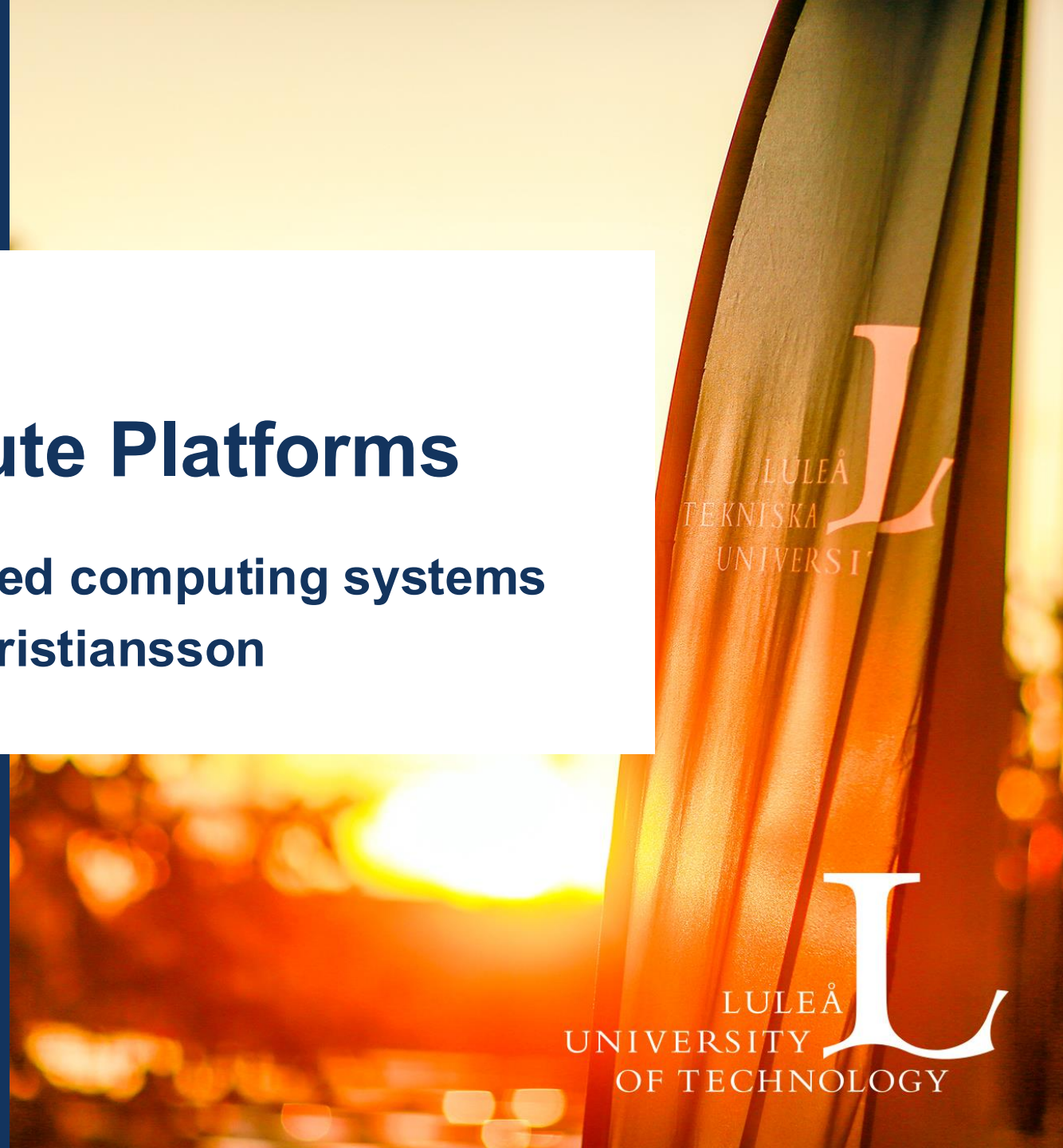
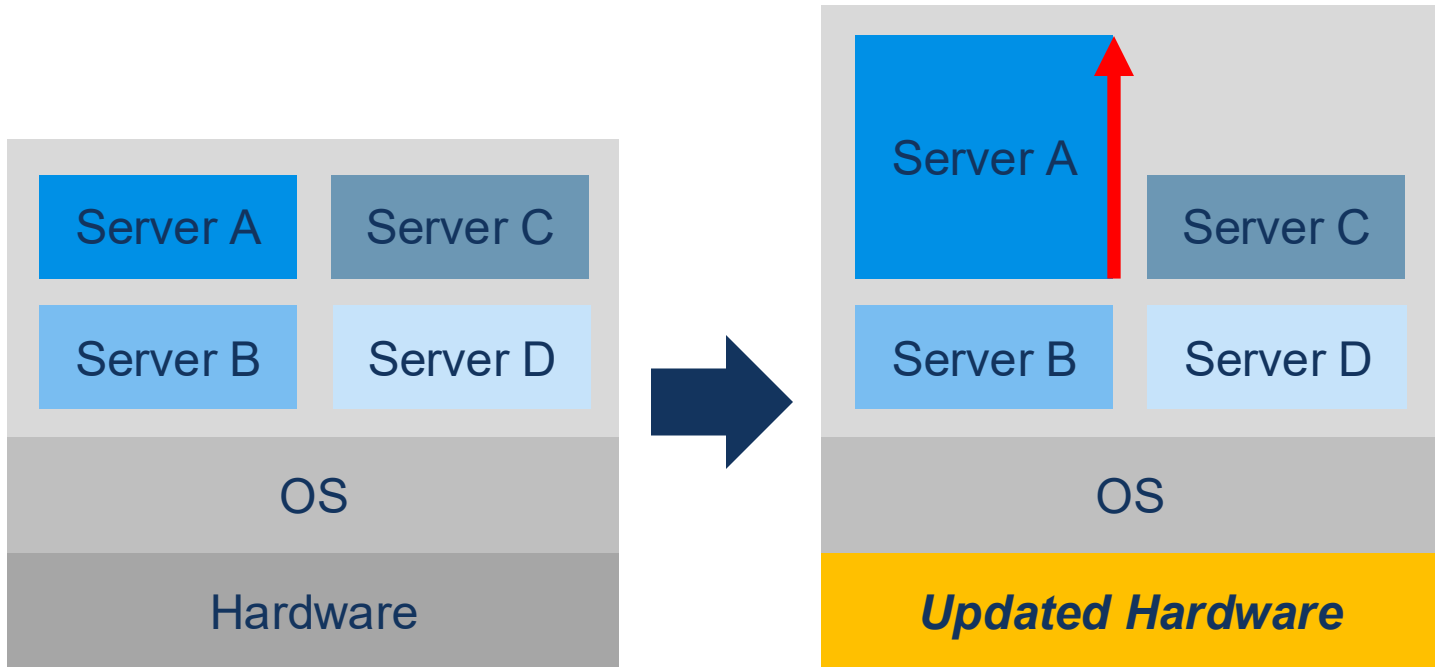
Modern Compute Platforms

Mobile and distributed computing systems

Johan Kristiansson



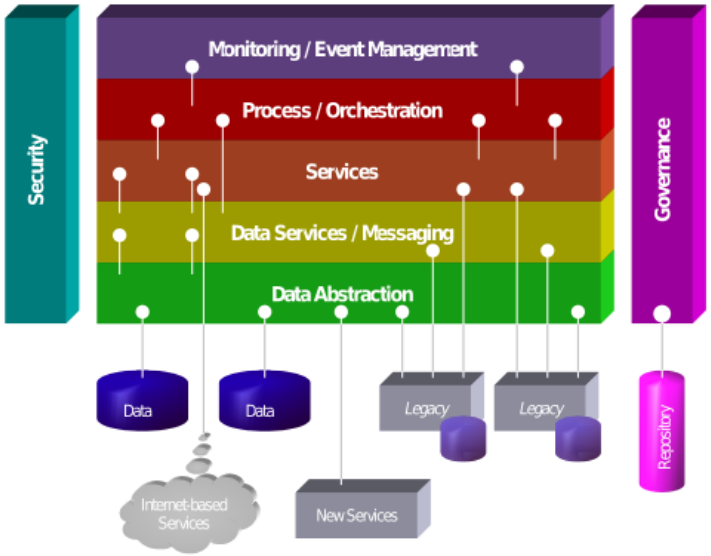
Pre-virtualisation Era (1990s-early 2000s)

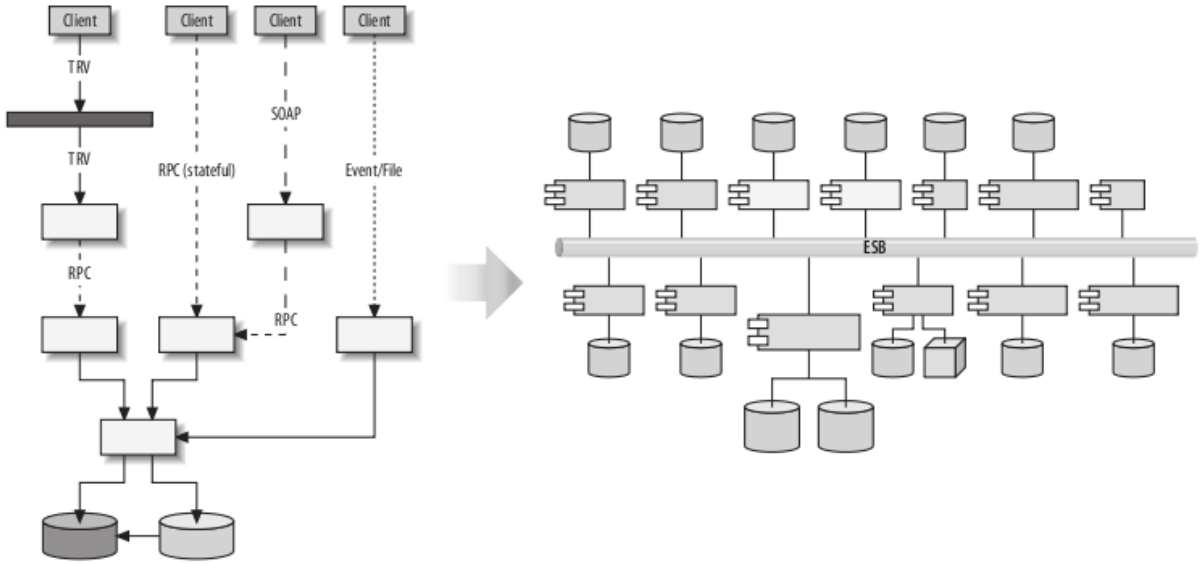


Server B need more memory, we need to buy an entire new physical server
We also need to re-install all new servers SW on the new server

Early 2000s-2012: Service-Oriented Architecture (SOA)

SOA is a way of designing software systems so that different parts of a system (called services) can interact with each other through well-defined interfaces, typically a service bus





Services can be flexibly orchestrated and reused to adapt to evolving business goals

- Loose coupling
- Interoperability
- Reusability
- Discoverability

Challenges with SOA

Complexity of Infrastructure

- SOA typically required a Service Bus (Enterprise Service Bus, or ESB) to manage communication between services, which sometimes became a bottleneck and a single point of failure
- Services in SOA often talk through XML-based protocols like SOAP, which are verbose and slow compared to lightweight alternatives (like REST or gRPC)

SOA services were usually deployed as Enterprise Archives (EARs) or Web Archives (WARs) on app servers like GlassFish or WebLogic

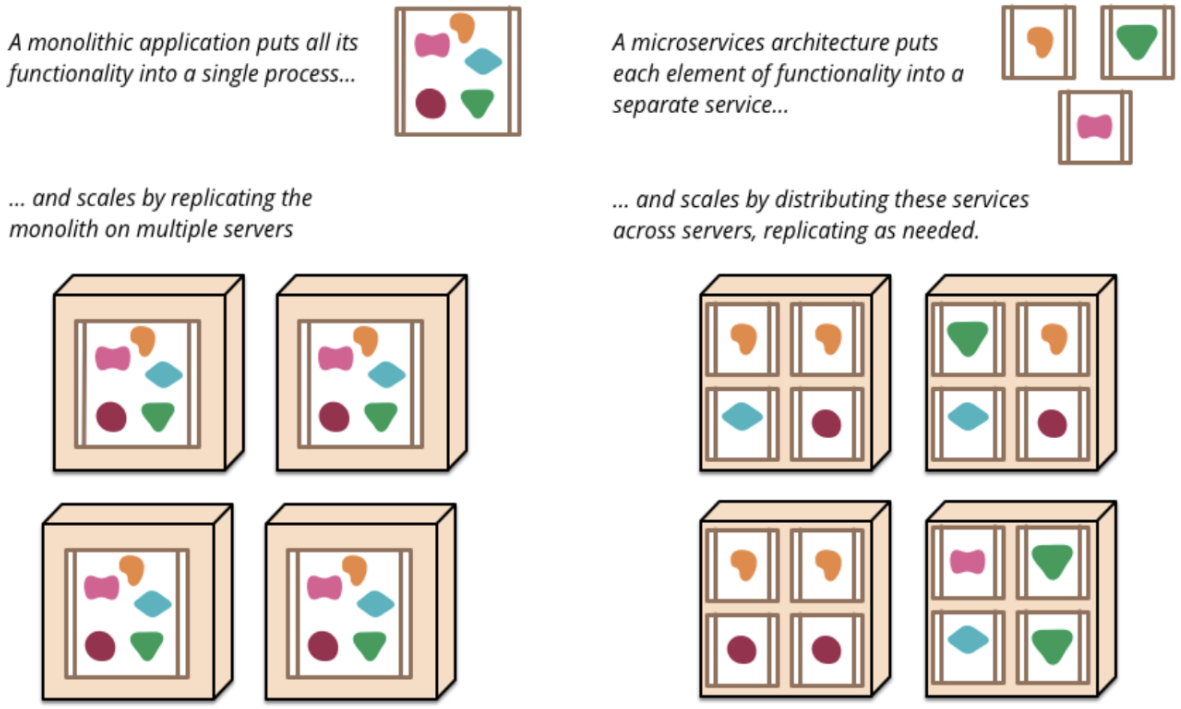
- Even though SOA promised modularity, most Java EE services were stateful, using EJBs tied to specific app servers and sessions (very hard to scale)
- **Not developer-friendly, steep learning curve (WSDL, XSD, EJB, WS-Security)!!!**

In practice, many SOA implementations turned into large, centralised, enterprise-wide projects dominated by Enterprise Service Buses (ESBs), ***massive integration hubs that glued together big monolithic systems***

Microservice (2014-2015)

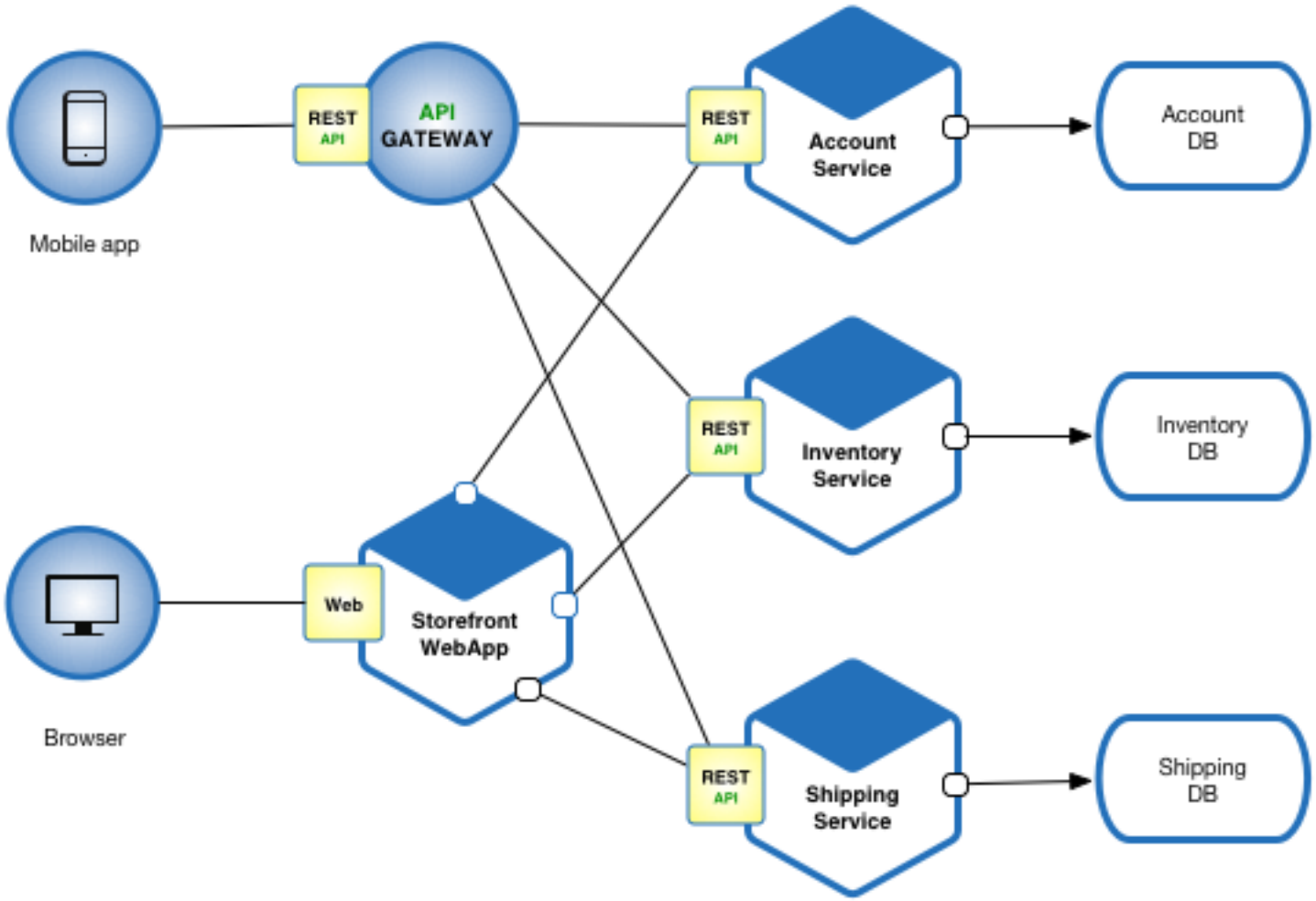
An approach to developing a single application as a suite of small services, each running in its own process and communicating with lightweight mechanisms and ***independently deployable by fully automated deployment machinery***

- Systems are **divided into small, deployable units** instead of shared libraries
- **Teams own their services** from development to operations
- Each service can use ***its own tools, languages, and database***
- Services ***manage their own data stores***, avoiding shared databases
- Systems anticipate and **recover from partial failures** (resilience)
- Continuous integration, deployment, and monitoring enable **fast, reliable delivery**
- Service boundaries **adapt as the system grows** and changes

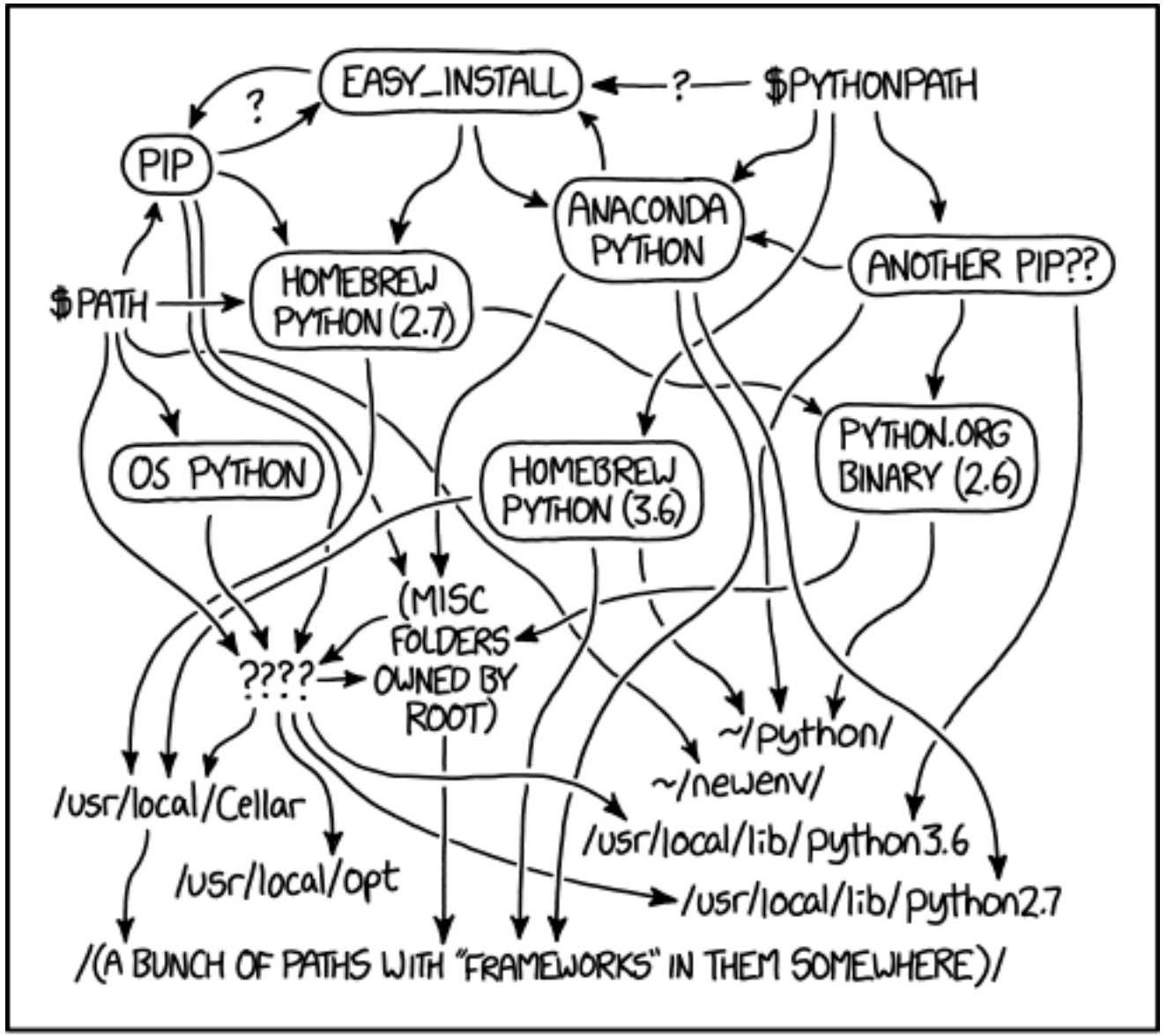


https://martinfowler.com/articles/microservices.html

Example of a Microservice Architecture



https://microservices.io/patterns/microservices.html



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.



- Loading/unloading was very time-consuming
- Inefficient transportation and storage
- Limited scalability
- Complex coordination
- Risk of damage and theft





“The storage itself needed to be transportable, not the goods within it”
In 1956 Malcolm McLean invented the shipping container

Container Era (2013 - Present)

Docker container is a shipping container for code

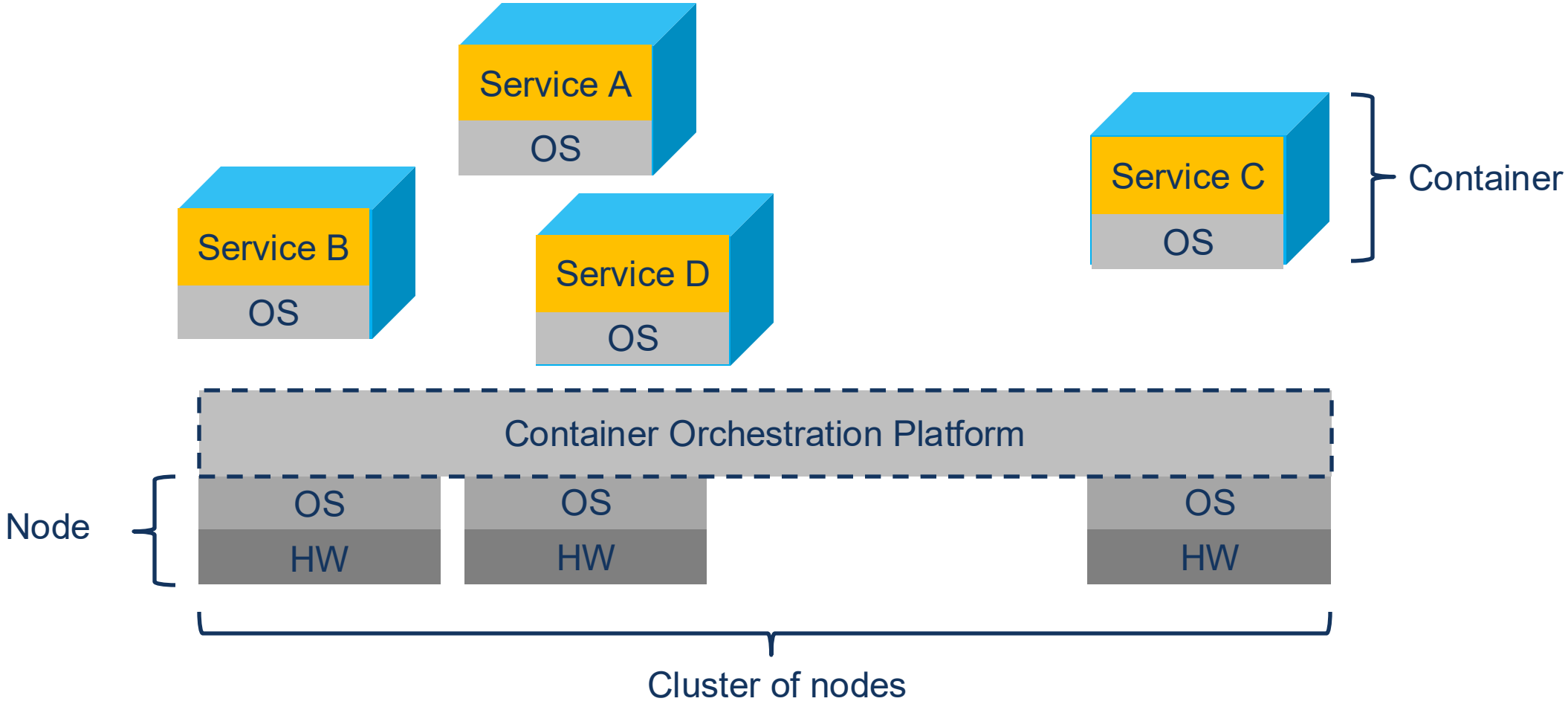


Docker was released in 2013

Containers package applications with their dependencies but share the host OS kernel, making them much lighter than VMs



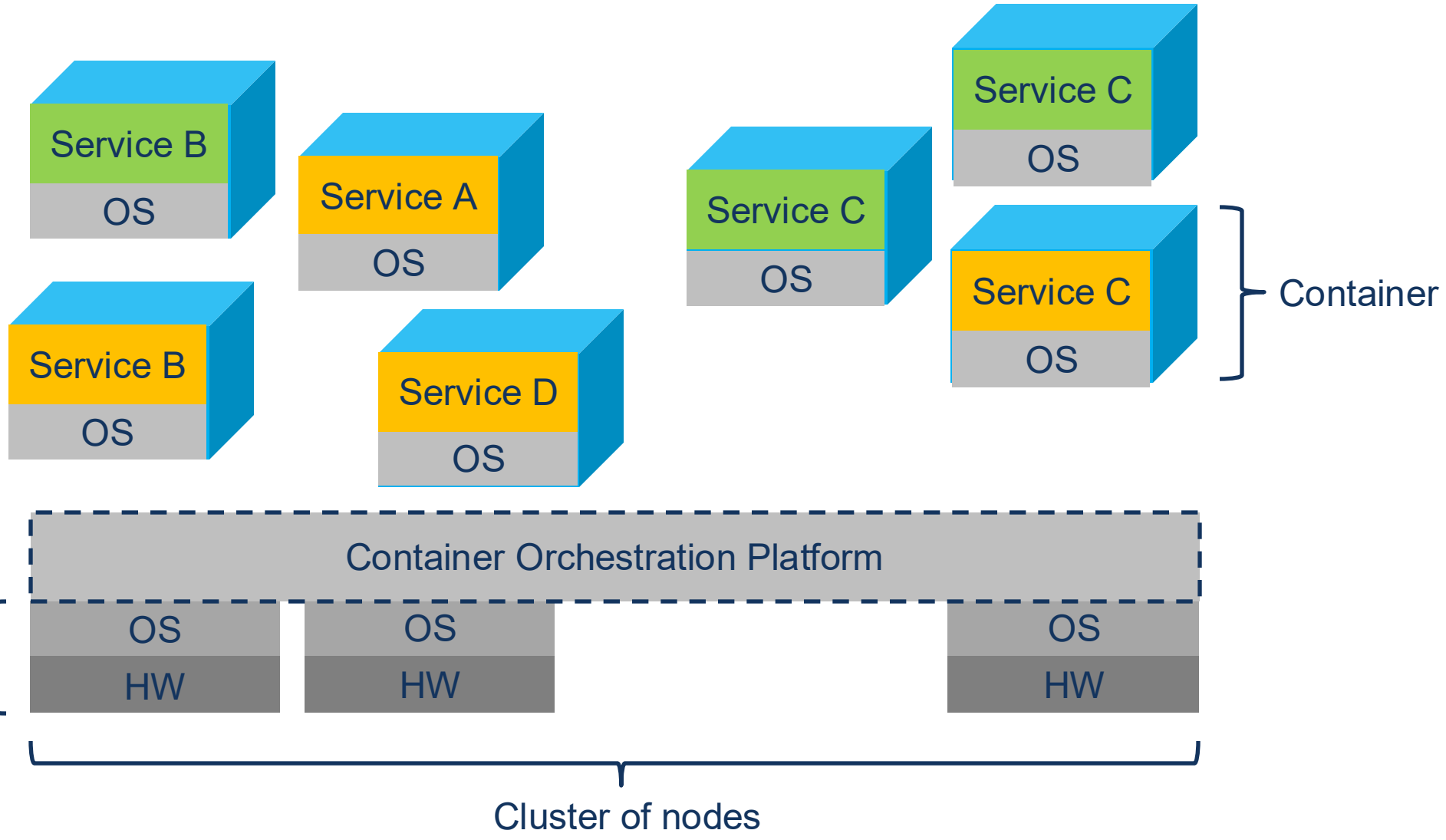
Container Era (2013 - Present)



Container Era (2013 - Present)

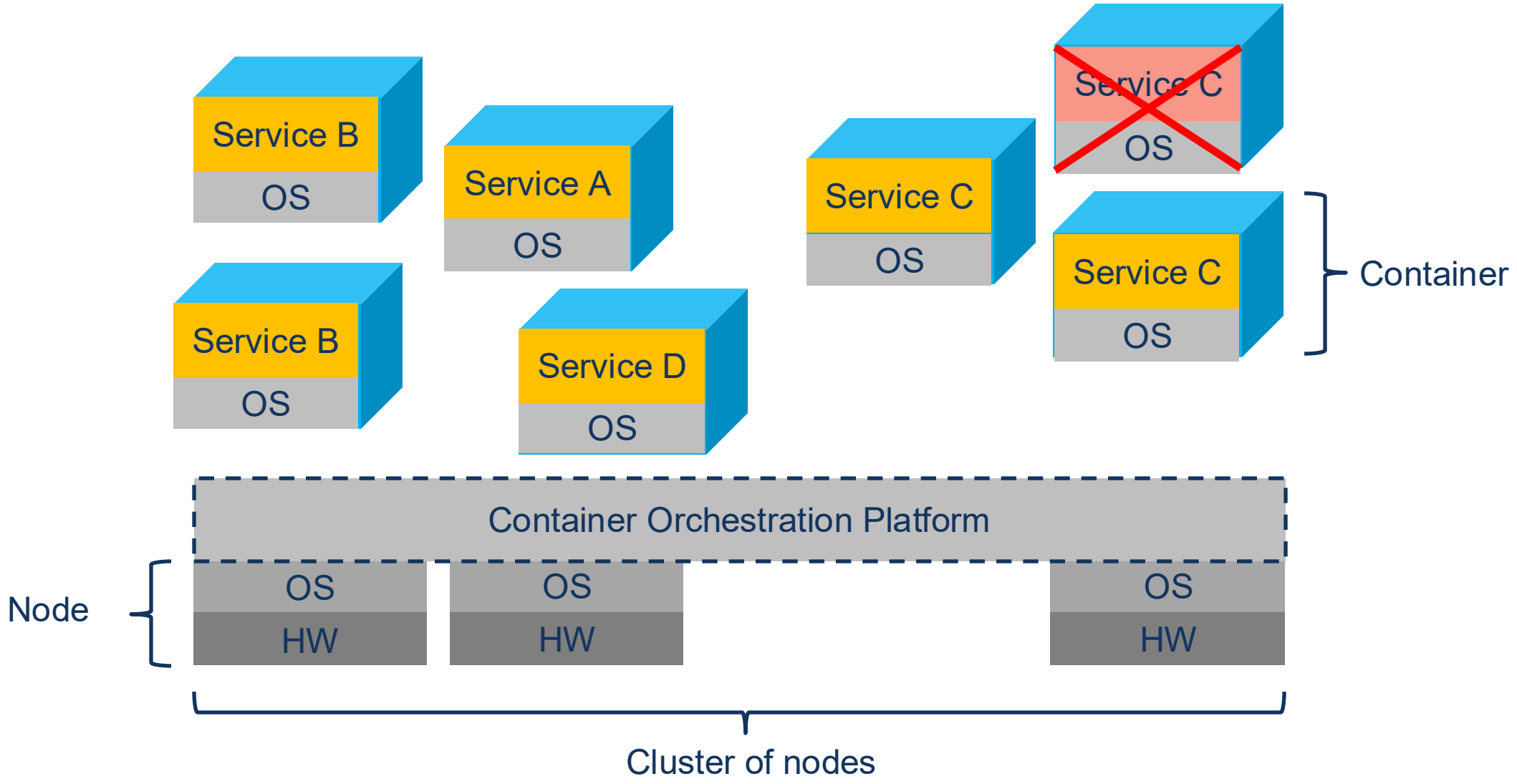
Applications can easily be scaled

As containers can be started almost instantly, we can create *elastic* (scale up/down) on-demand applications

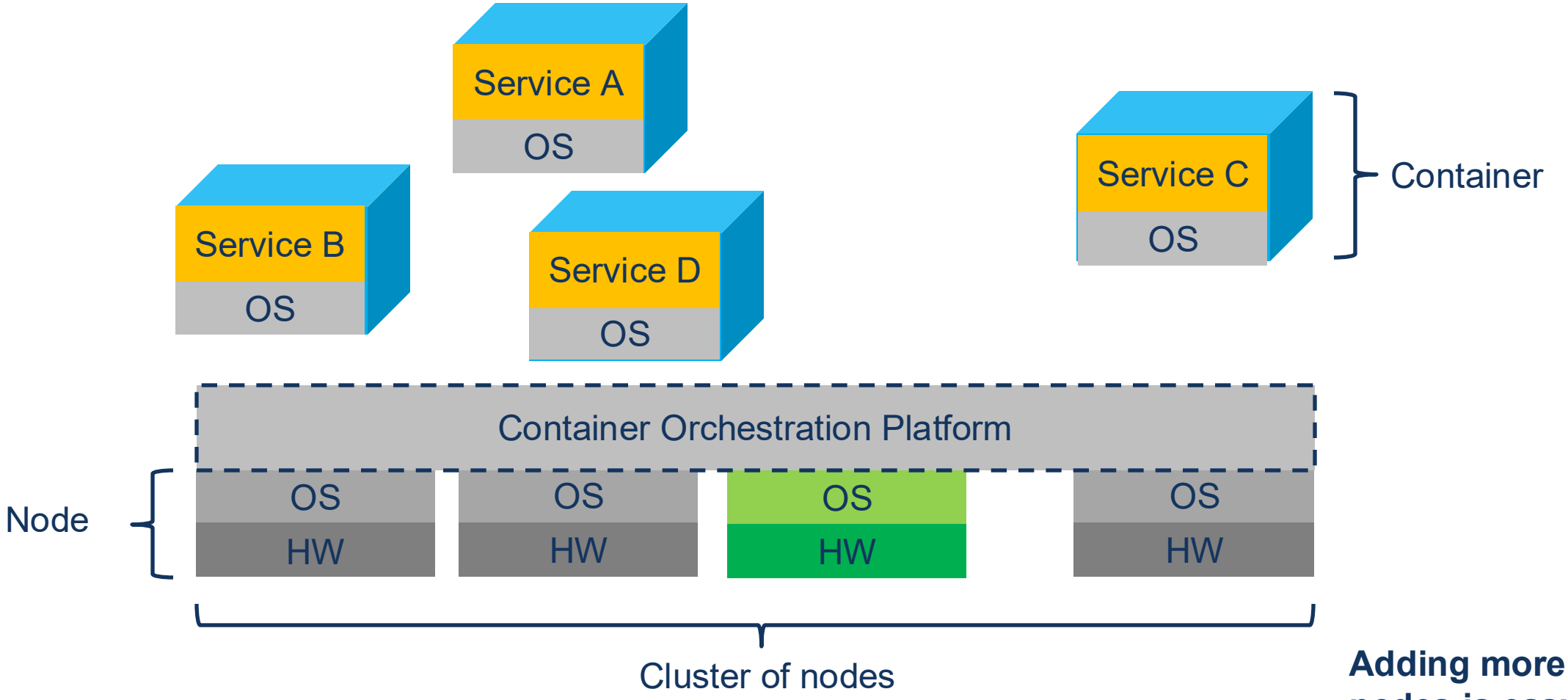


Container Era (2013 - Present)

If a container crashes, other containers may not be affected



Container Era (2013 - Present)



Adding more nodes is easy


kubernetes
11004
4+
11004
@datamattsson
IDEAS INCLUDED
Pod
Service
StatefulSet
DaemonSet
Deployment

Kubernetes is an open-source system for ***automating the deployment, scaling***, and management of containerised applications

It was originally designed by Google, based on their internal system Borg, now maintained by the Cloud Native Computing Foundation (CNCF)



Features:

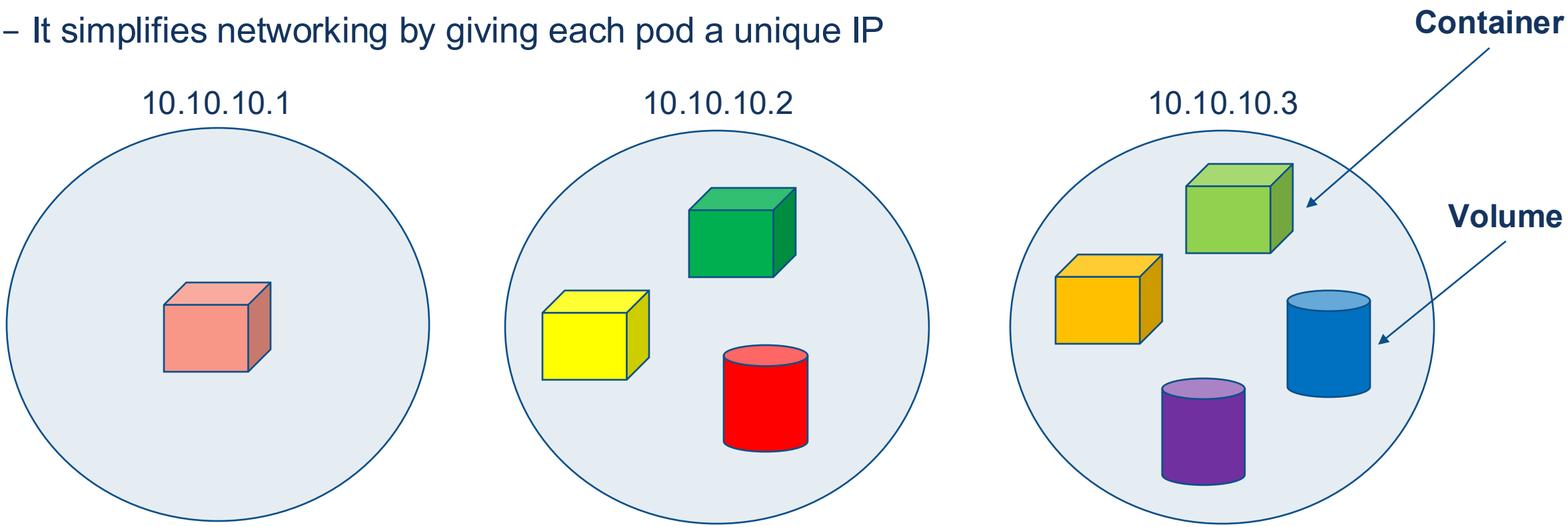
- Container orchestration
- Self-healing
- Scaling
- Load balancing
- Service discovery
- Declarative configurations
- Immutable infrastructure

One of the central tools in modern microservice architectures

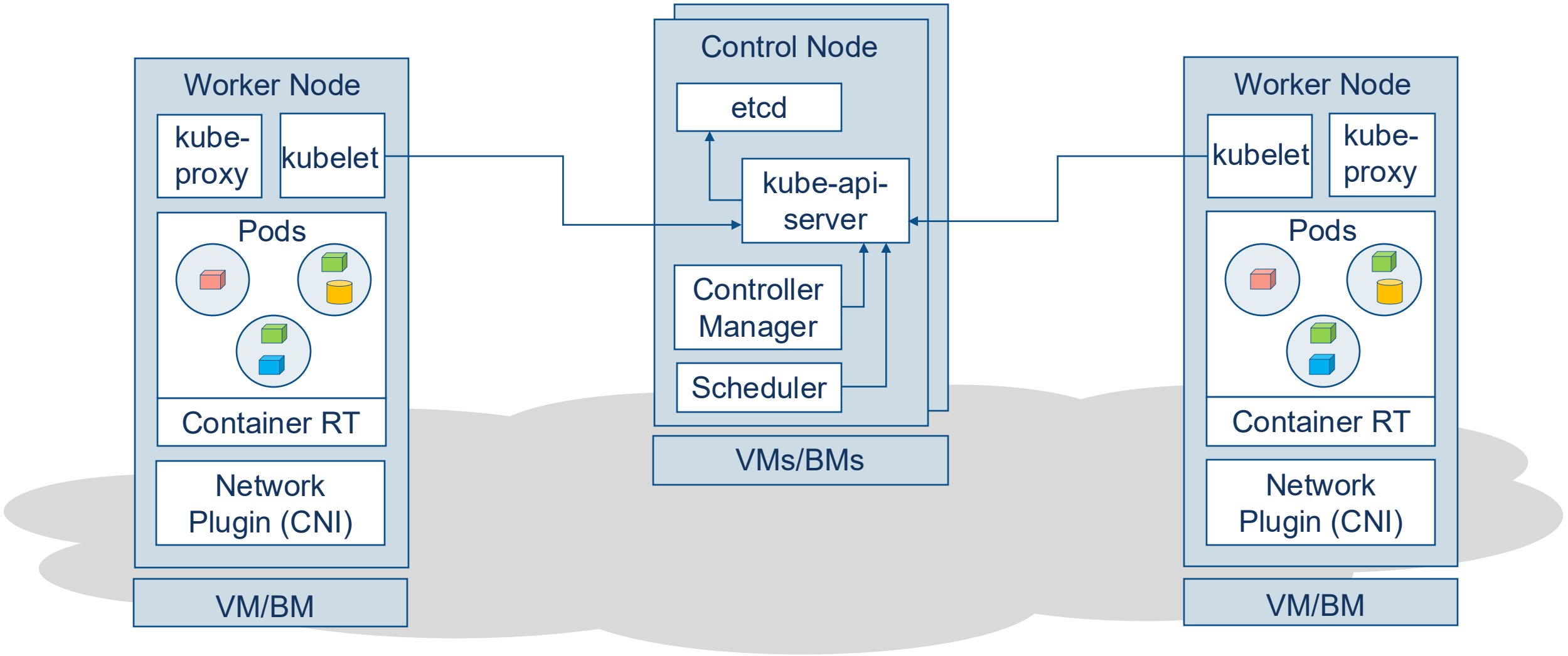
Pods

A Kubernetes Pod is the smallest and most basic deployable unit in Kubernetes

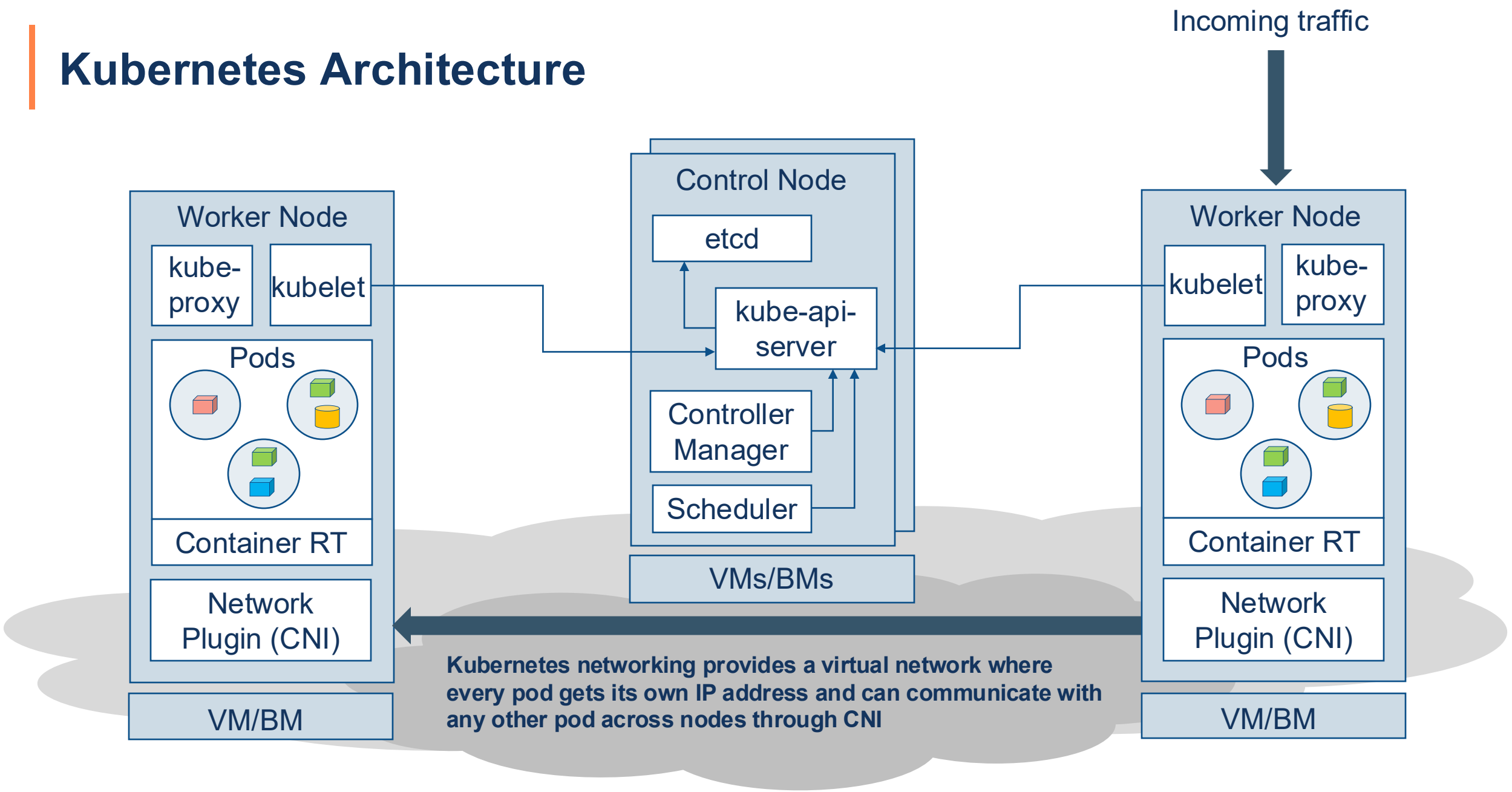
- It allows for patterns where tightly coupled containers need to work together
- It provides a consistent unit for scheduling and resource allocation
- It simplifies networking by giving each pod a unique IP



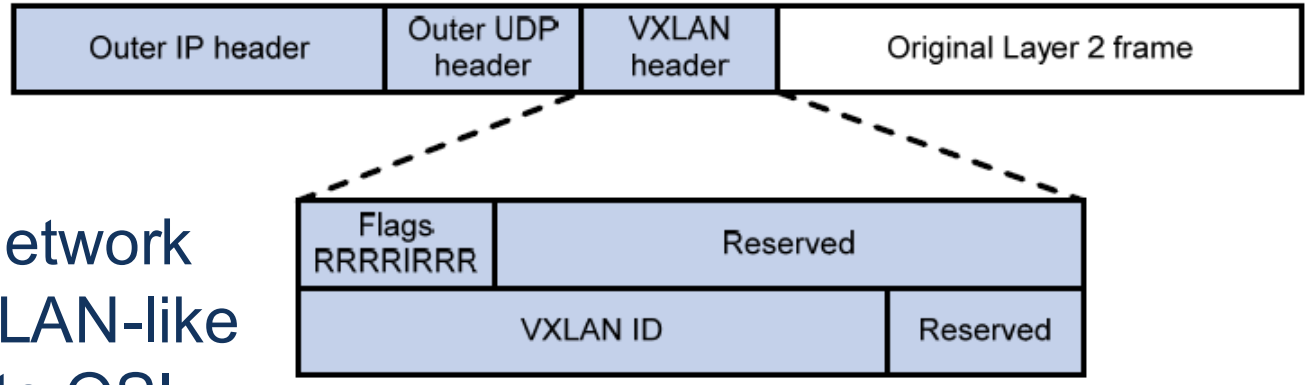
Kubernetes Architecture



Kubernetes Architecture



Network Virtualisation



Virtual eXtensible LAN (VXLAN) is a network virtualisation technology that uses a VLAN-like encapsulation technique to encapsulate OSI layer 2 Ethernet frames within layer 4 UDP datagrams

A Pod sends an Ethernet frame to another Pod:

1. The node's CNI plugin (Flannel) grabs that frame
2. It wraps it up in a VXLAN packet
3. It sends that packet across the underlying IP network to the destination node
4. The destination node unwraps it and delivers the original Ethernet frame to the target Pod.

Container Network Interface

Flannel

- Simple overlay network (VXLAN)
- Easy to deploy and maintain, minimal configuration
- No Network Policies support
- Ideal for small or development clusters

Calico

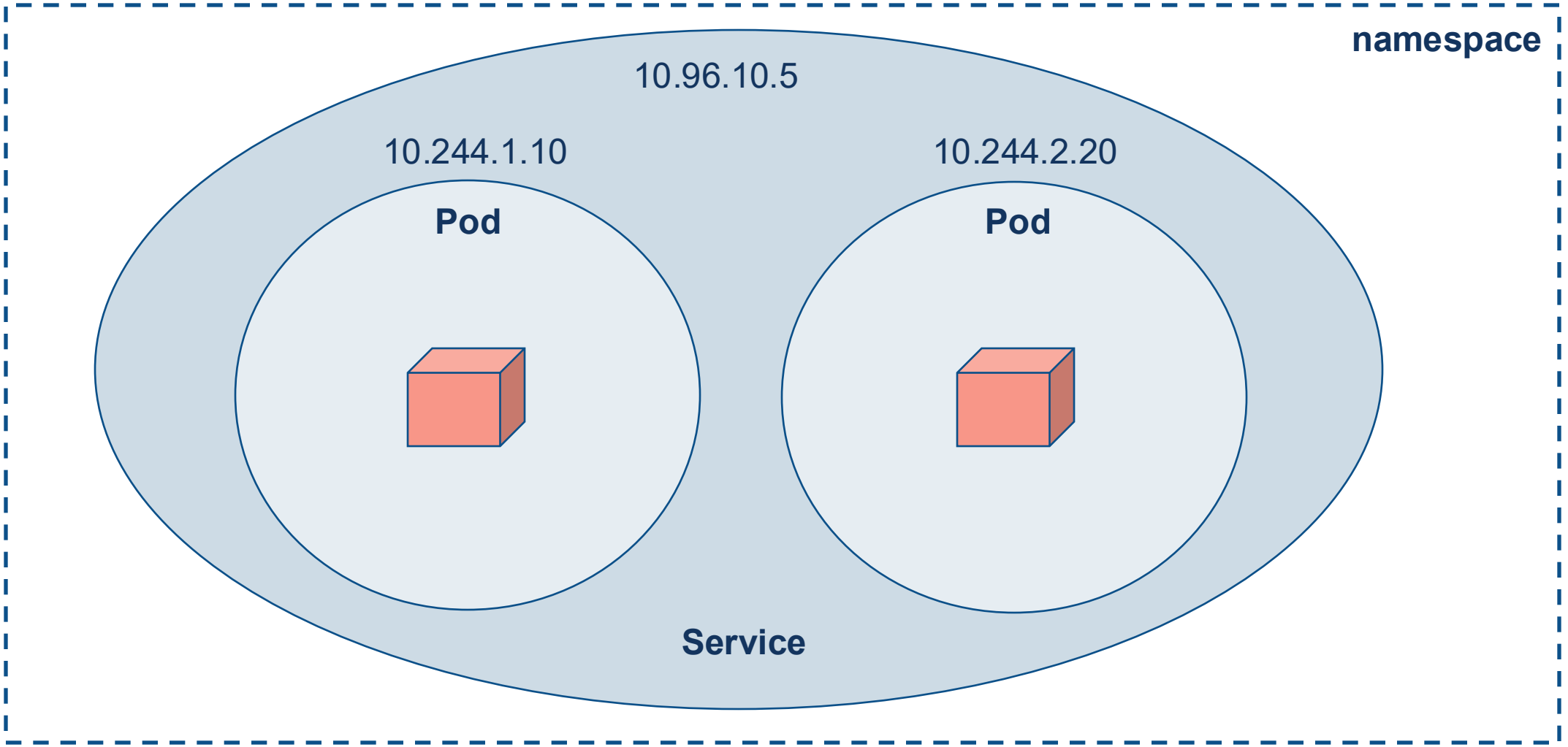
- Routing-based (Layer 3) networking
- Supports Kubernetes Network Policies and advanced security features

Cilium

- Built on eBPF (kernel-level packet processing
- Excellent performance; bypasses iptables bottlenecks

Services

Kubernetes Services implement load balancing by using a virtual ClusterIP that is mapped by kube-proxy (via iptables/IPVS) or eBPF programs to dynamically rewrite packet destinations at the kernel level, distributing connections across Pod endpoints



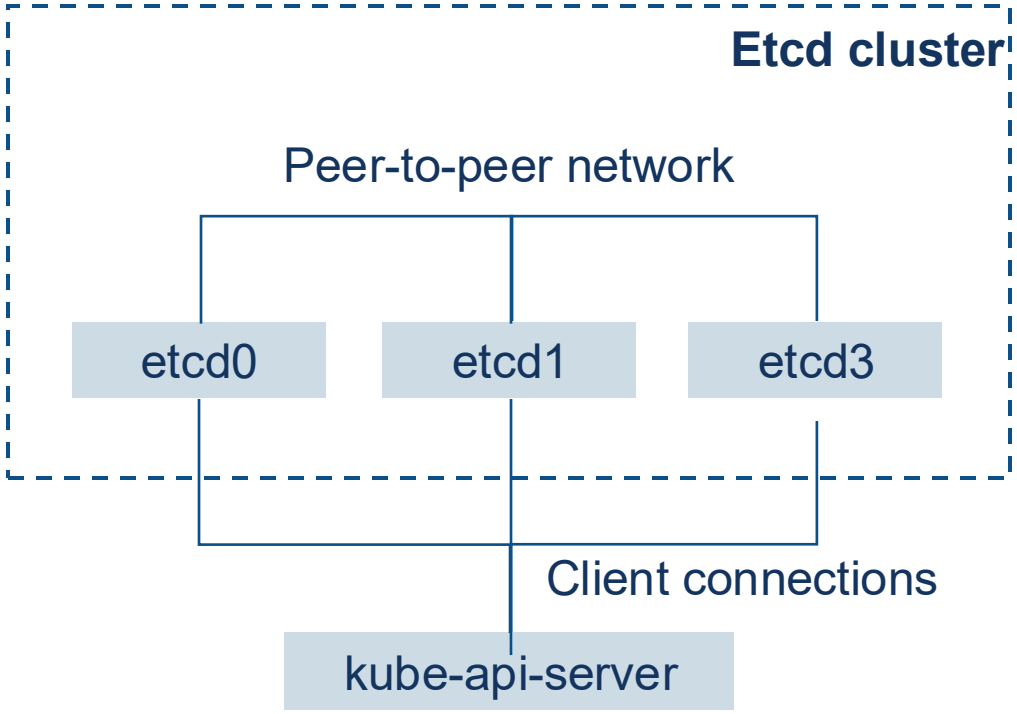
Etcd

A distributed, consistent key-value database that stores all Kubernetes cluster state (Pods, ConfigMaps, Secrets, etc.)

- Acts as the single source of truth for the desired and actual cluster state

Uses the Raft consensus algorithm to ensure data consistency across multiple nodes, even during failures

- Only one scheduler decision per Pod
- No duplicate assignments
- Controllers, schedulers, and API servers must all operate on the same view of cluster state



Kubernetes Controllers

Kubernetes is declarative, not imperative

- You don’t tell it how to do something, you tell it what you wan

Controllers in Kubernetes are control loops that constantly watch and adjust the cluster’s state toward a desired state

In Kubernetes, everything runs on the principle of ***desired state vs. actual state***

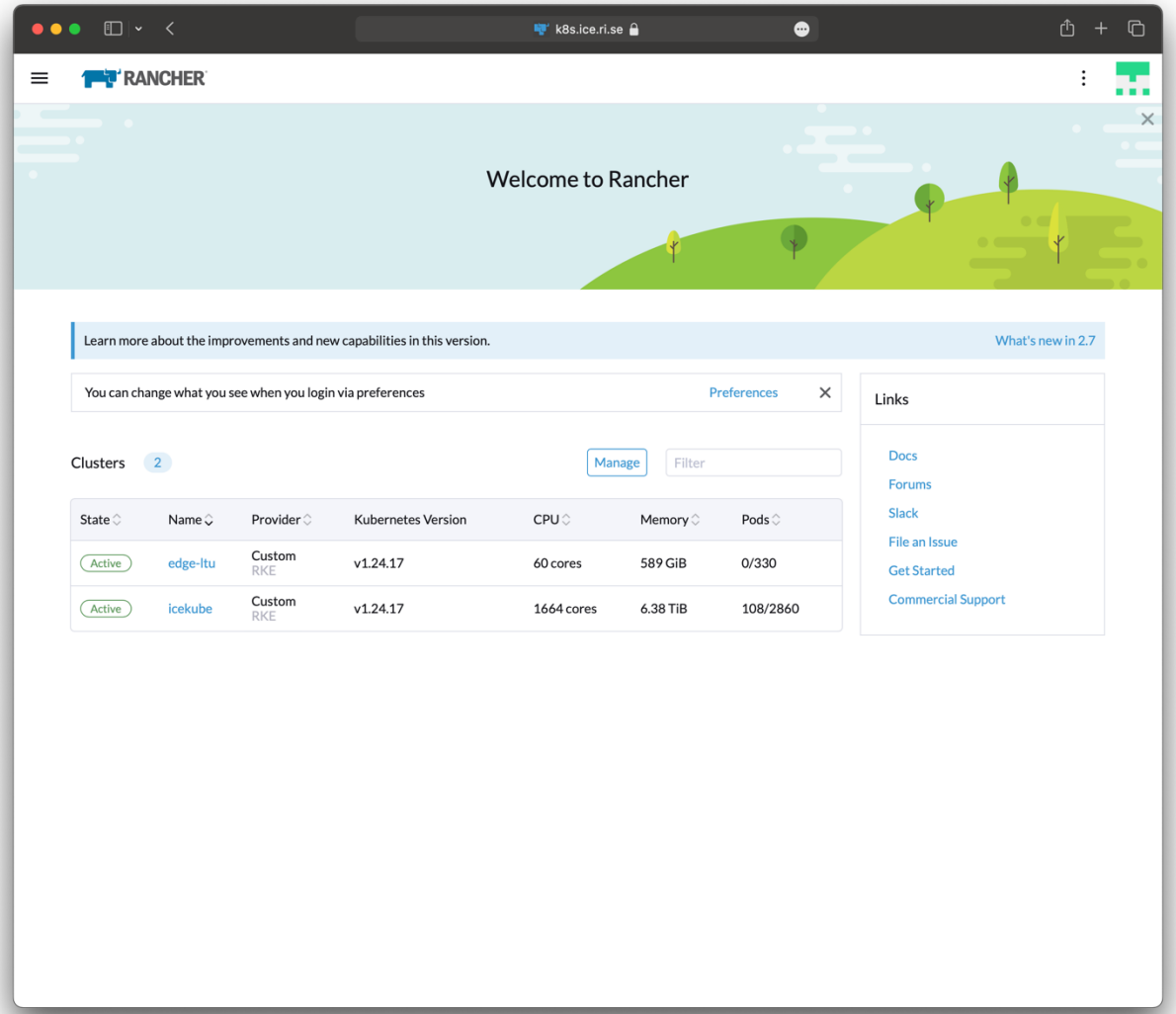
Each controller in Kubernetes performs a ***reconciliation loop***

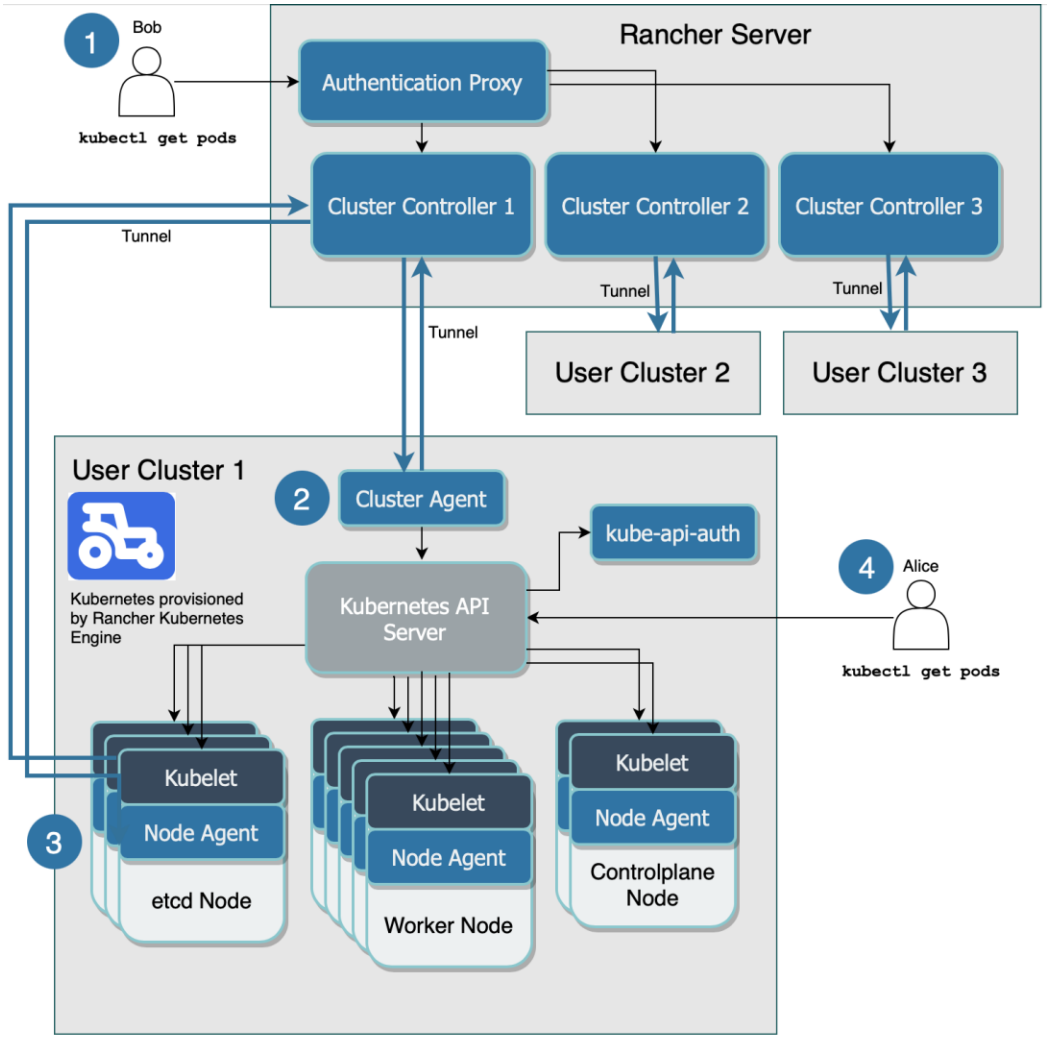
1. **Observe**
   It watches the cluster’s actual state through the API server (e.g. what Pods are running?)
2. **Compare**
   It reads your resource’s desired state from its spec (the user wants three replicas)
3. **Act**
   If there’s a difference, makes changes to bring reality closer to desire
   e.g. only two Pods exist? Start one more.
4. **Repeat forever**

Kubernetes Controllers

- Replication Controller
  - Ensures the right number of Pods exist
- Job Controller
  - Watches Jobs and creates Pods that run once and finish
- Deployment Controller
  - Manages rolling updates and rollbacks of Deployments
- Node Controller
  - Monitors the health of worker nodes, marks nodes as NotReady if they stop reporting and evicts Pods from failed nodes
- Ingress Controller
  - Decides how incoming HTTP or HTTPS requests get forwarded to your internal Services and Pods

Rancher – Kubernetes Management Platform





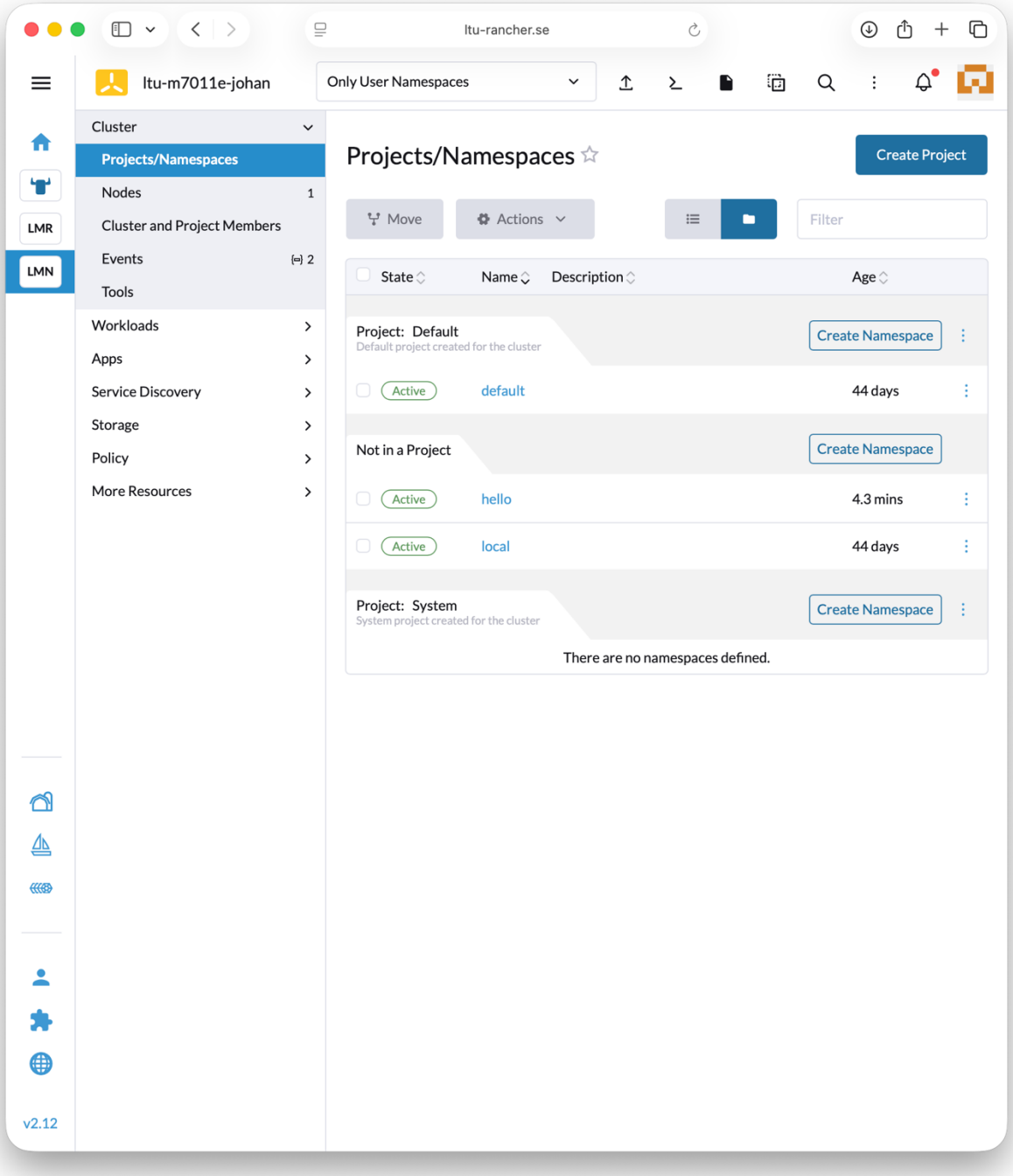
Kubernetes Helloworld Example

$ kubectl create namespace hello

namespace/hello created

$ kubectl get namespaces

```
NAME                          STATUS   AGE
cattle-fleet-system           Active   44d
cattle-impersonation-system   Active   44d
cattle-system                 Active   44d
cert-manager                  Active   44d
default                       Active   44d
hello                         Active   32s
kube-node-lease               Active   44d
kube-public                   Active   44d
kube-system                   Active   44d
local                         Active   44d
```



Kubernetes Helloworld Example

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  labels:
    app: hello-world
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-world
        image: nginx
        ports:
        - containerPort: 80
```

Resources (e.g. Pods, Services, Deployments) use **labels** and **selectors** for indirection, just like DNS maps domain names to IP addresses, **selectors** map to resources by matching labels instead of hardcoded identifiers

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  type: NodePort
  selector:
    app: hello-world
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
```

Kubernetes Helloworld Example

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  labels:
    app: hello-world
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-world
        image: nginx
        ports:
        - containerPort: 80
```

service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  type: NodePort
  selector:
    app: hello-world
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
```

The port on the container (nginx listens on port 80)

The port on the node that you access externally

The port on the node that you access externally: range 30000-32767

Kubernetes Helloworld Example

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  labels:
    app: hello-world
spec:
  replicas: 1
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: hello-world
        image: nginx
        ports:
        - containerPort: 80
```

$ kubectl apply -f deployment.yaml --namespace hello

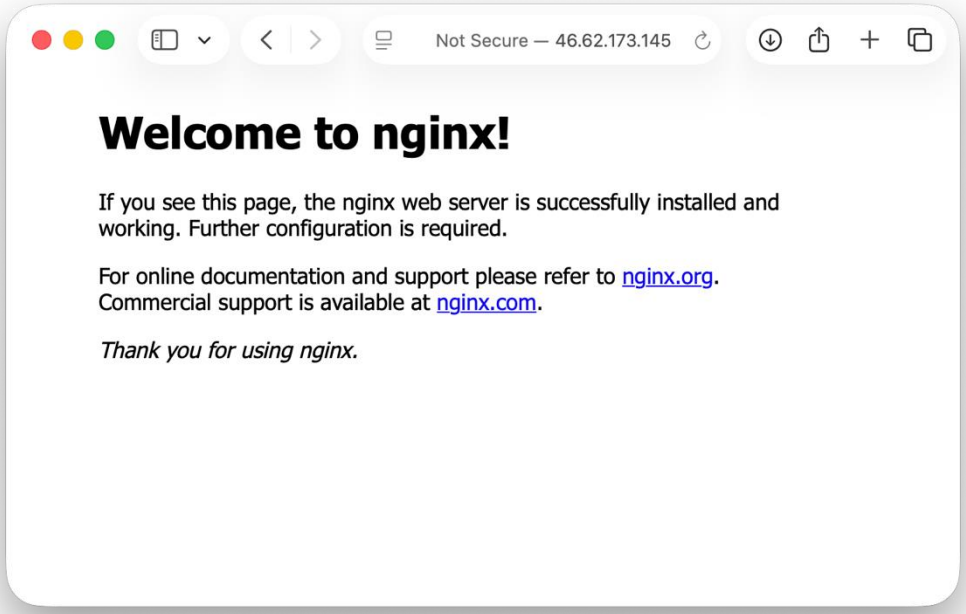
deployment.apps/hello-world created

$ kubectl apply -f service.yaml --namespace hello

service/hello-world-service created

service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  type: NodePort
  selector:
    app: hello-world
  ports:
  - protocol: TCP
    port: 80
    targetPort: 80
    nodePort: 30080
```



Let’s build a Microservice

app.py

```
from flask import Flask

app = Flask(__name__)

@app.route('/')
def hello():
    return '<h1>Hello World from Python!</h1>'

@app.route('/health')
def health():
    return {'status': 'healthy'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN pip install flask

COPY app.py .

EXPOSE 8080

CMD ["python", "app.py"]
```

$ docker build -t johan/helloworld-test .

```
Sending build context to Docker daemon  3.072kB
Step 1/6 : FROM python:3.11-slim
3.11-slim: Pulling from library/python
8c7716127147: Pull complete
44350d10c02e: Pull complete
4dc2c3222cdb: Pull complete
b25238518c0c: Pull complete
….
Successfully built 3a2702a26e26
Successfully tagged johan/helloworld-test:latest
```

$ docker push johan/helloworld-test

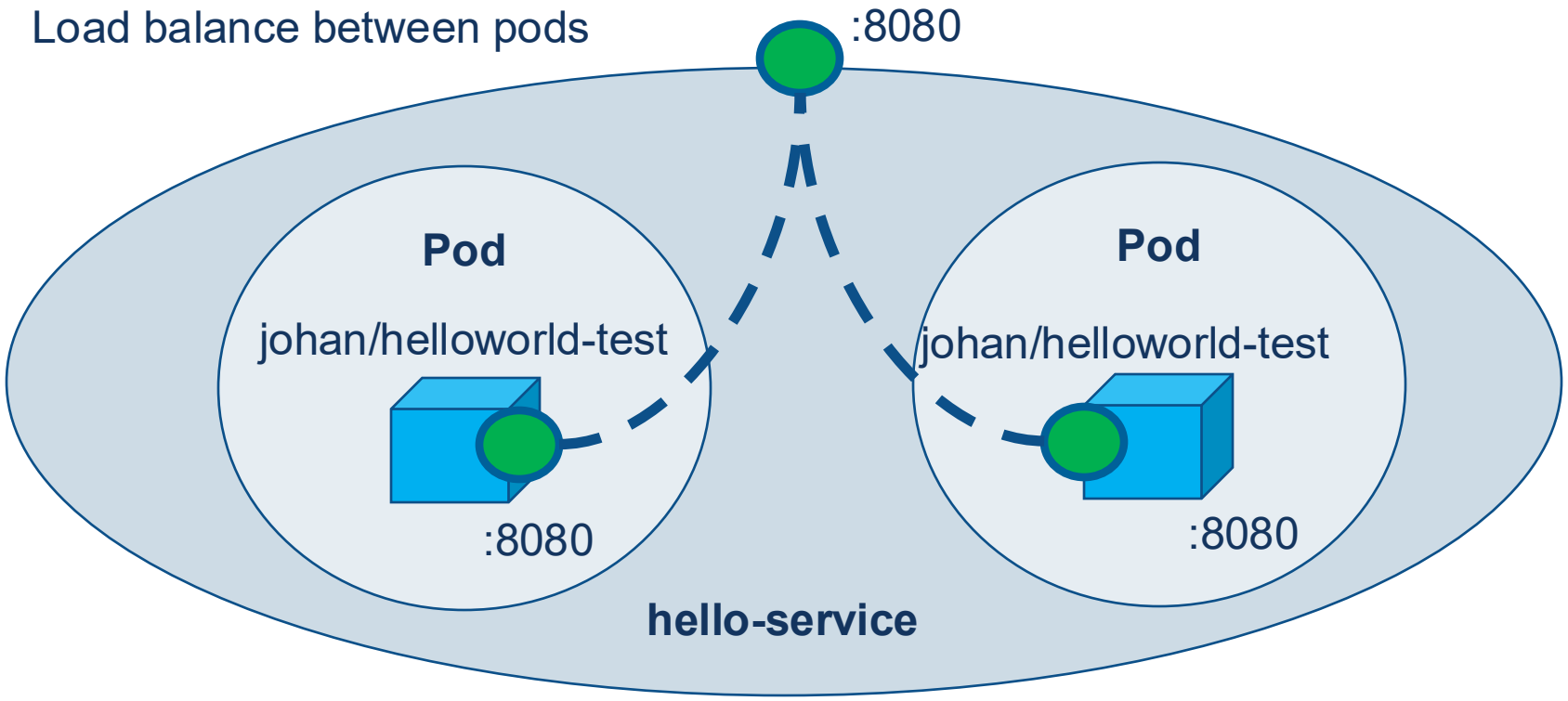
```
Using default tag: latest
The push refers to repository [docker.io/johan/helloworld-test]
2eacb79abdda: Pushed
1f1475bb02ba: Pushed
0cea68289b4a: Pushed
3ecbc03a21ab: Pushed
53ced85d0e60: Pushed
60b14e839bd0: Pushed
1d46119d249f: Pushed
latest: digest:
sha256:d42d6183eeaeda827be3f52d897d88eb5509a962a222bf5521d69d
72b7b5457d size: 1783
```

Let’s build a Microservice

$ docker run -p 8080:8080 johan/helloworld-test:latest

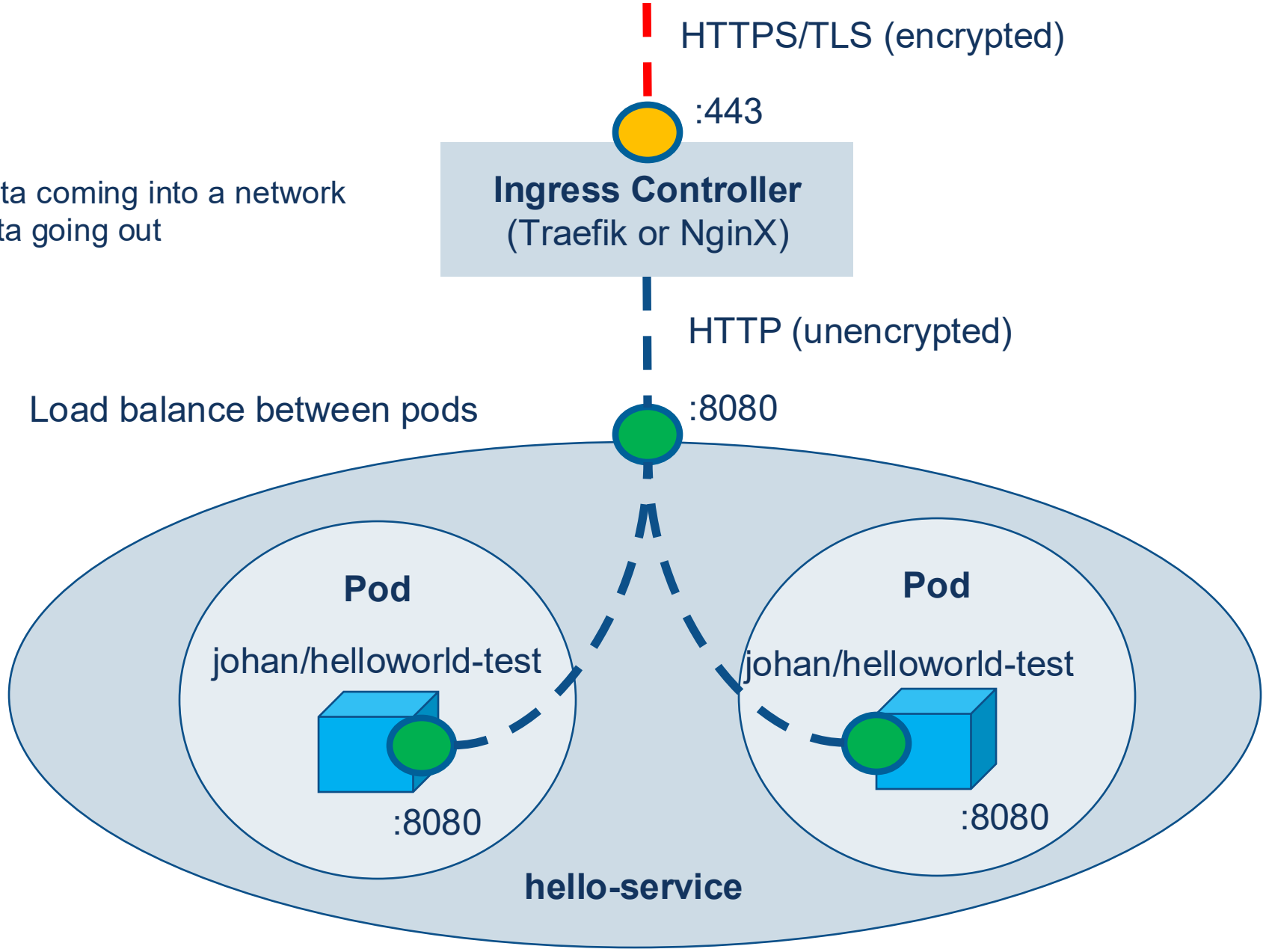
$ curl localhost:8080
<h1>Hello World from Python!</h1>

Services

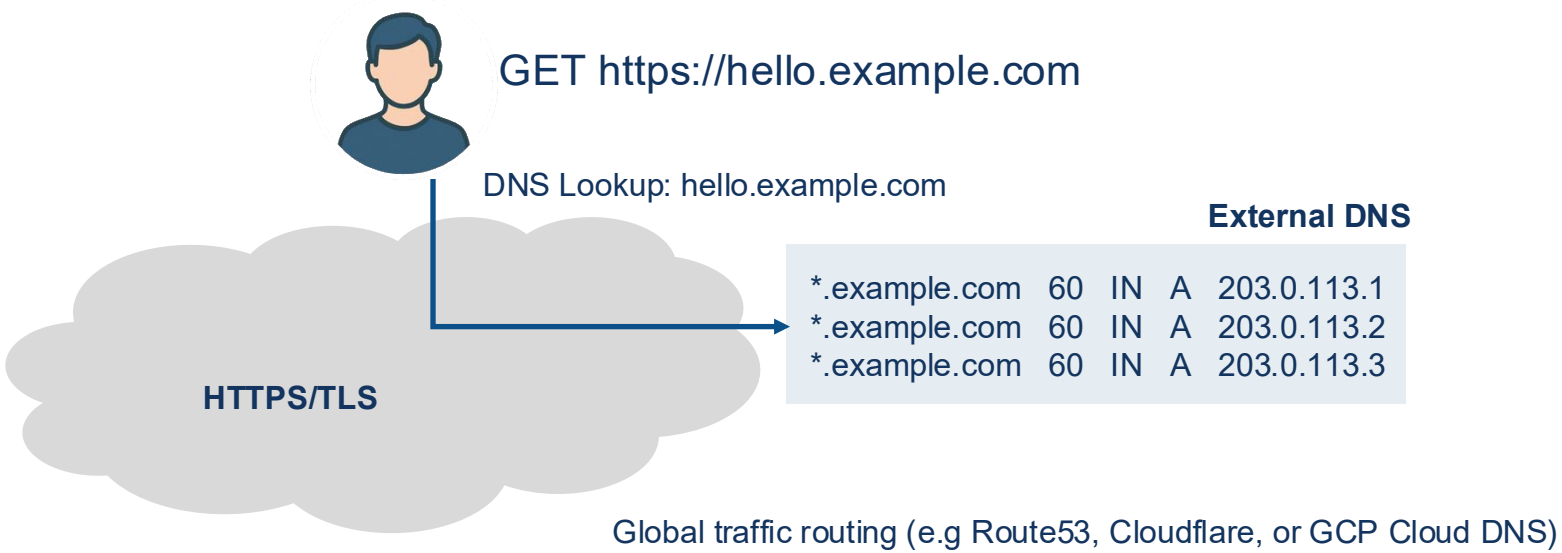

Load balance between pods
:8080
Pod
johan/helloworld-test
:8080
Pod
johan/helloworld-test
:8080
hello-service

Ingress

Ingress traffic is data coming into a network
Egress traffic is data going out

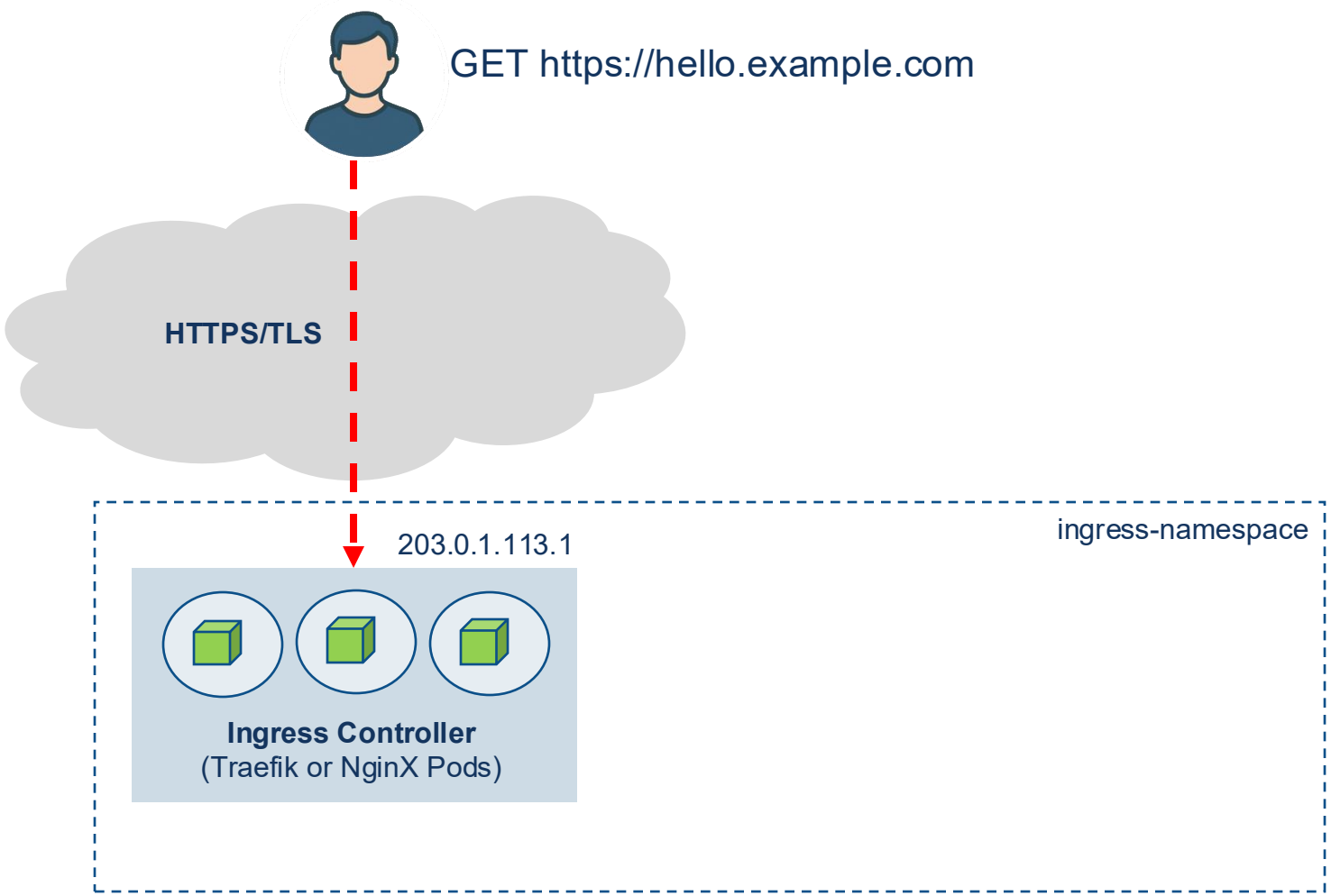


Ingress Controller

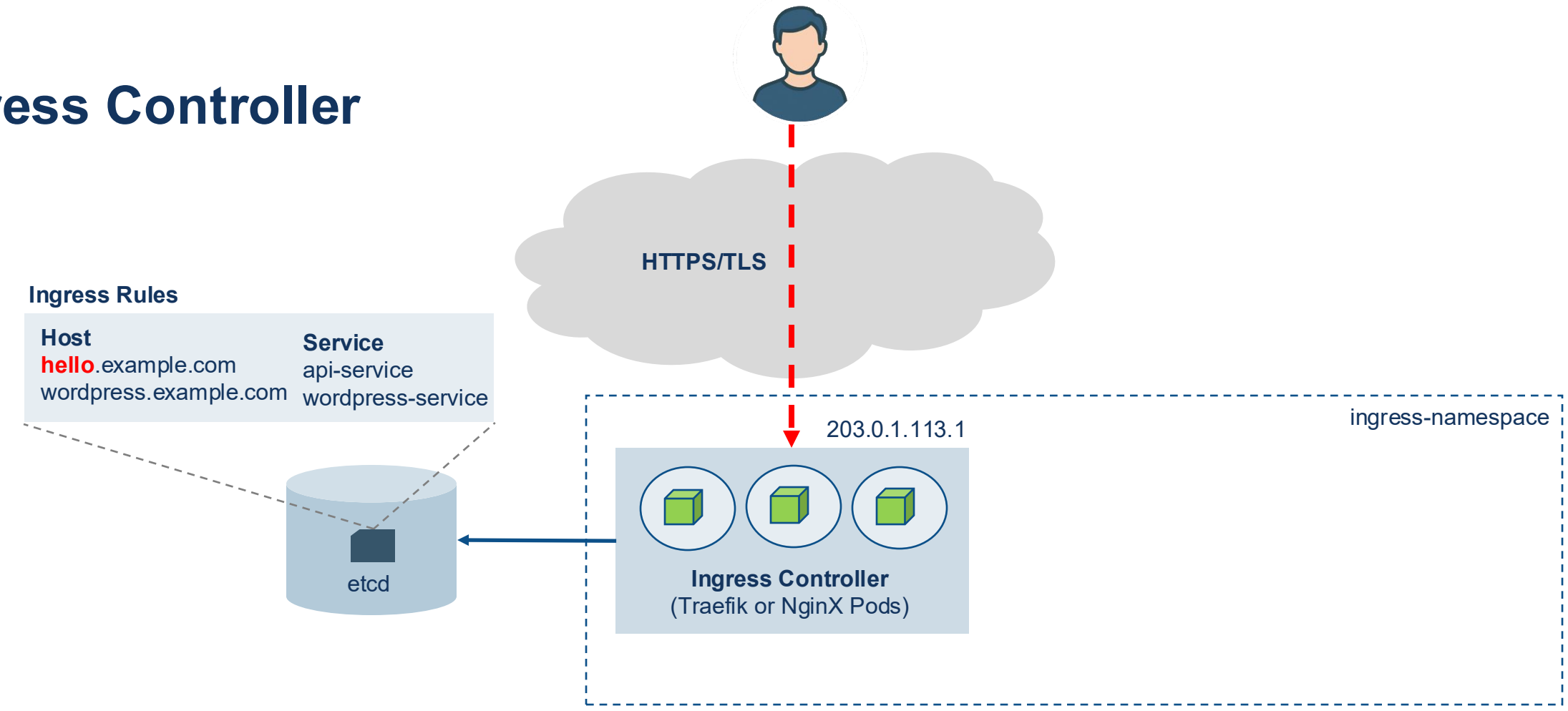


Global traffic routing (e.g Route53, Cloudflare, or GCP Cloud DNS)

Ingress Controller

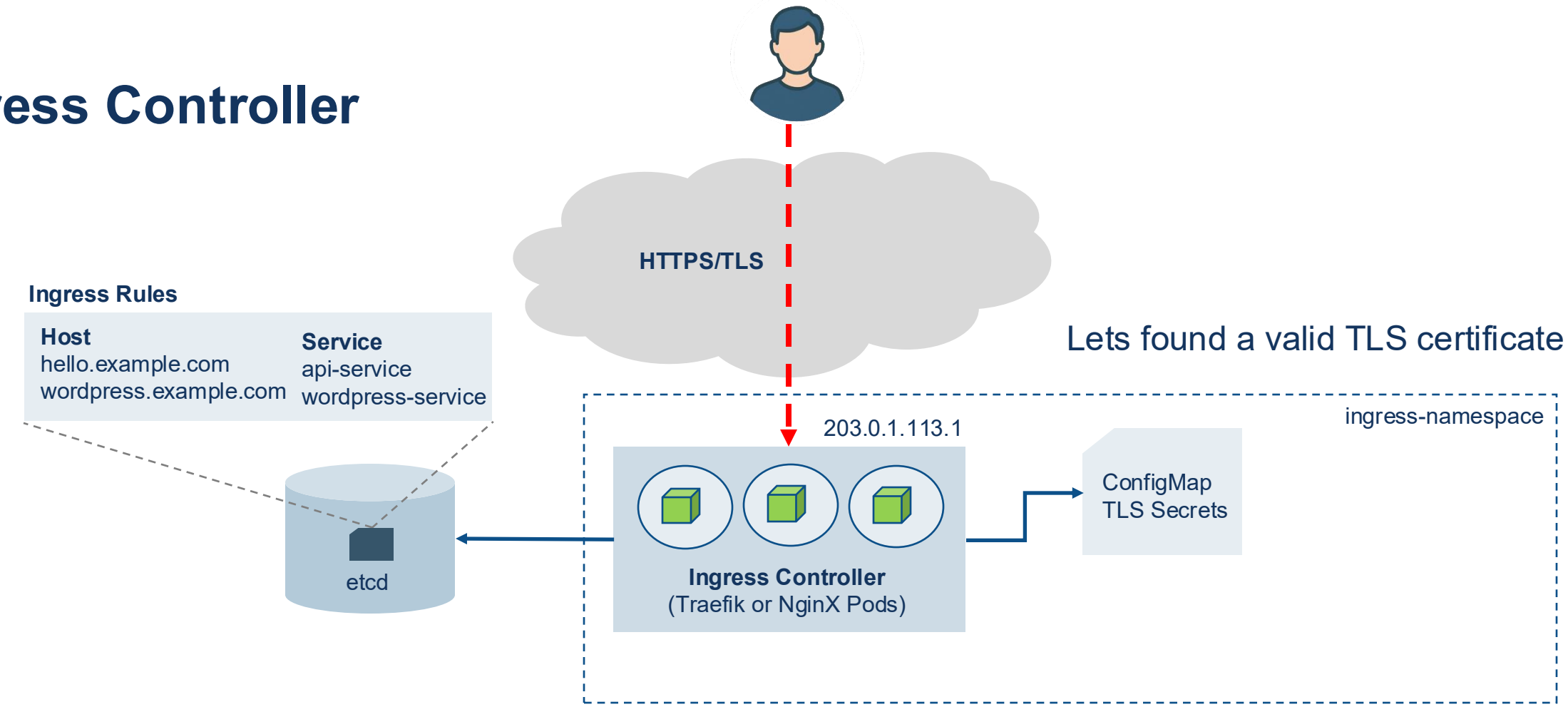


Ingress Controller

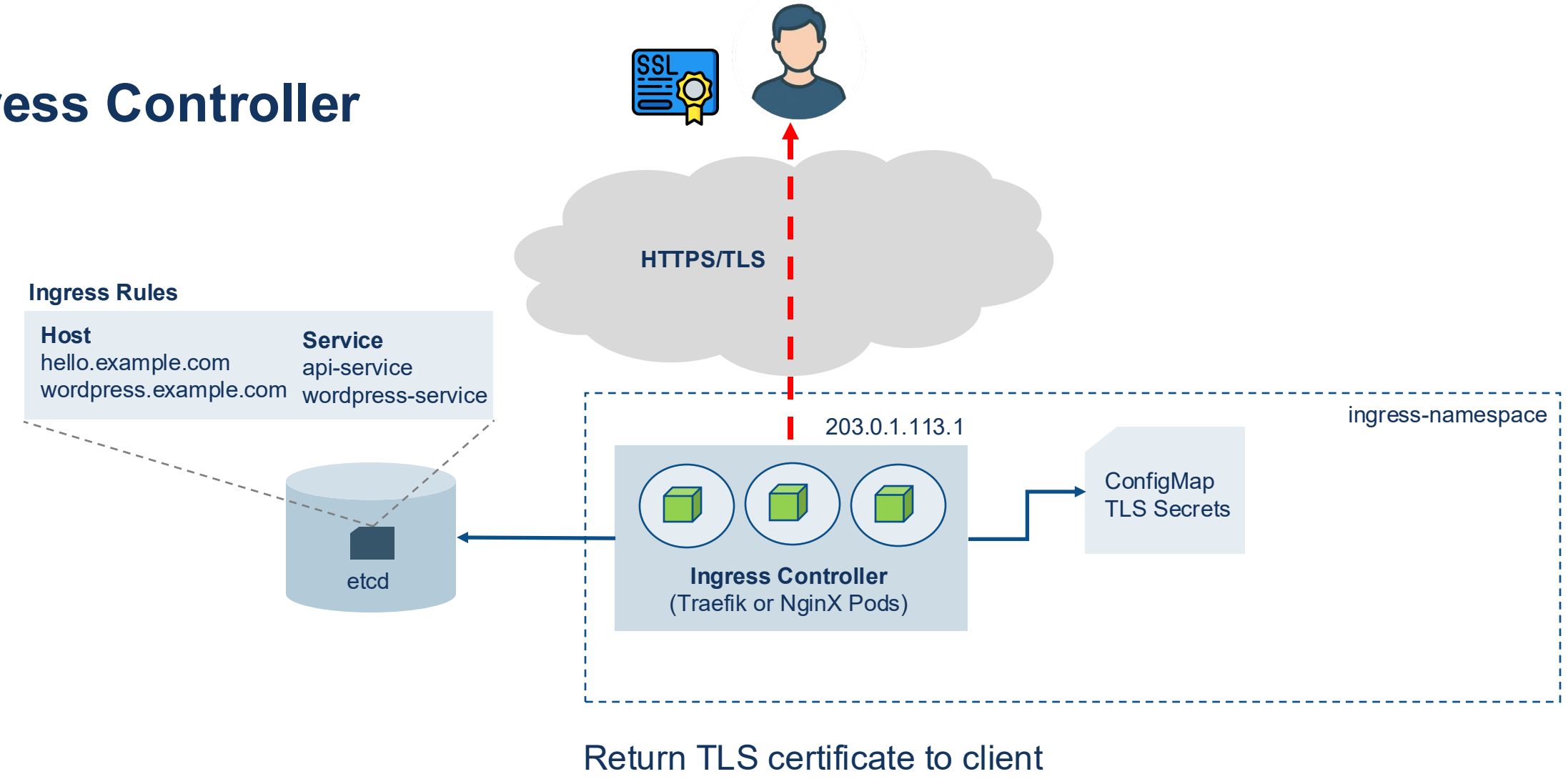


Got a HTTP Request: https://**hello**.example:443

Ingress Controller

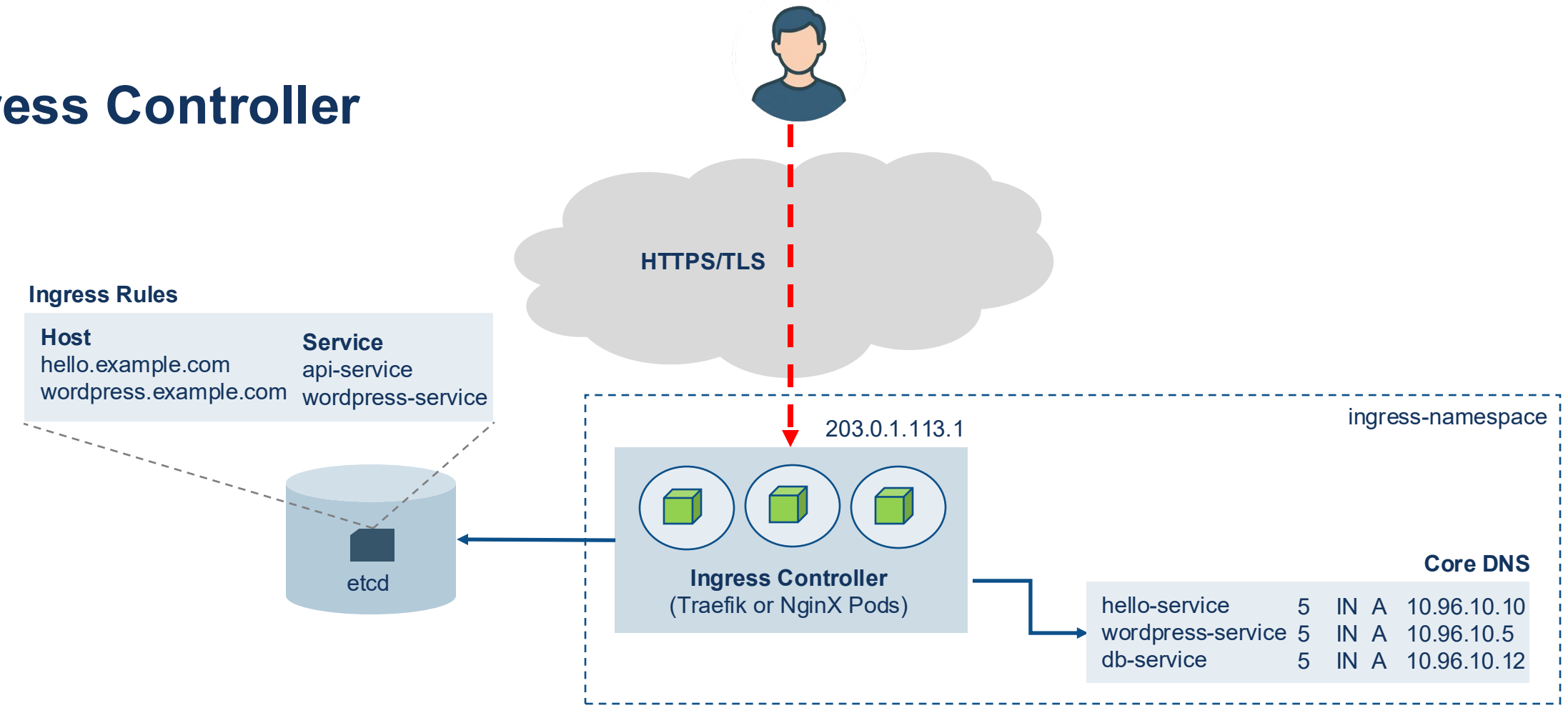


Ingress Controller



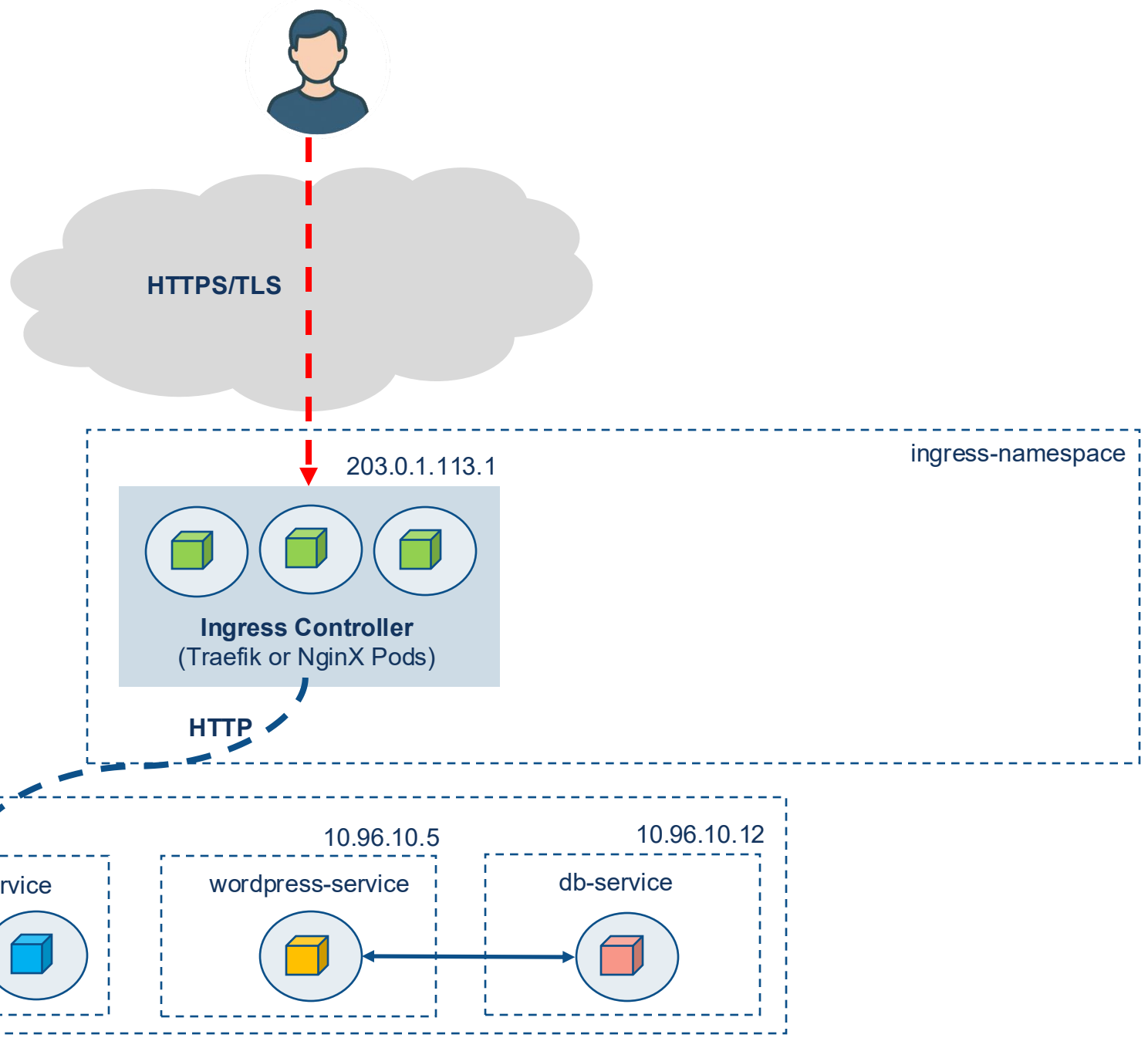
Return TLS certificate to client

Ingress Controller

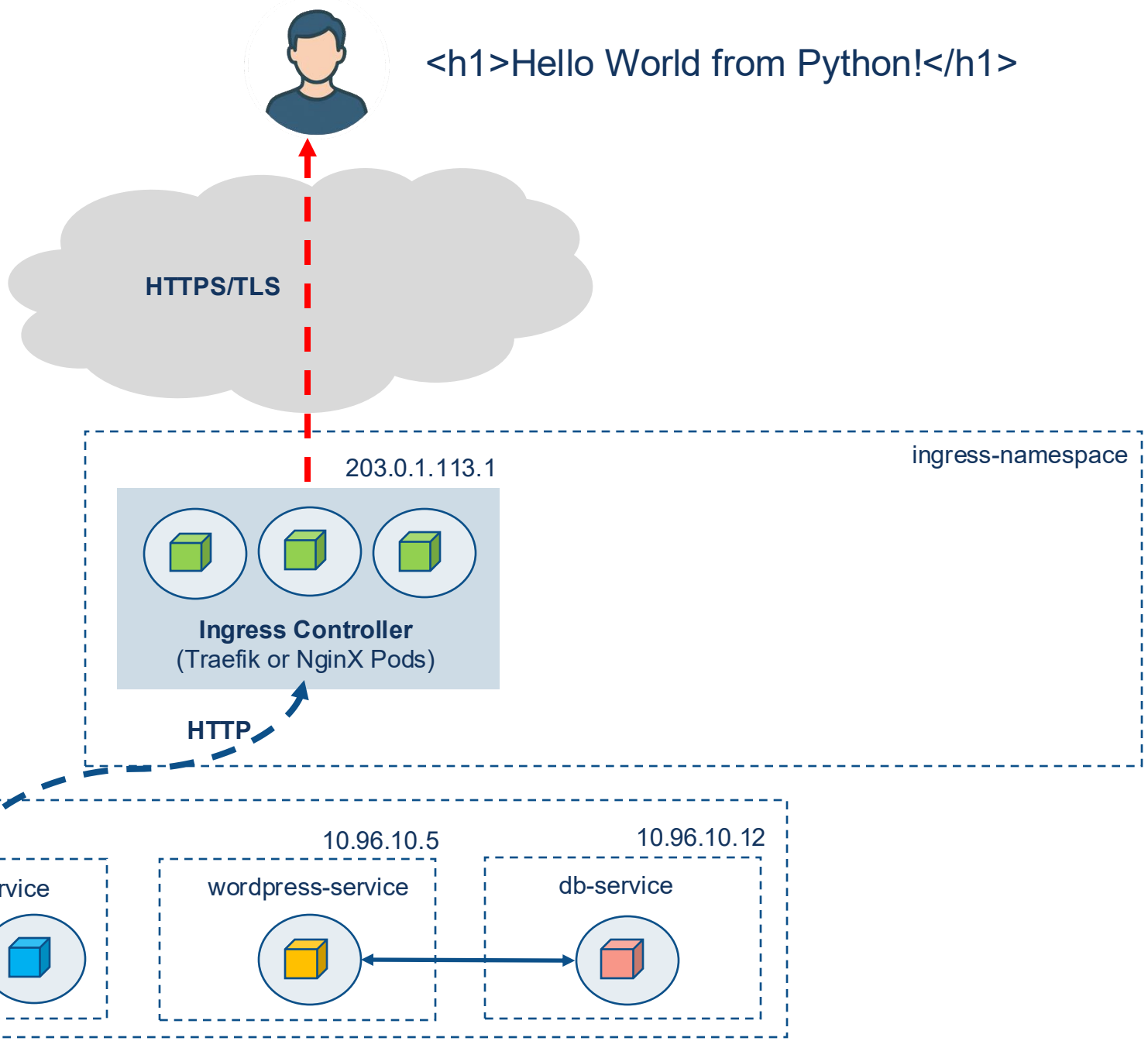


Terminte HTTPS connectpom and forward traffic to corresponding service using HTTP
Lookup IP to service using built-in DNS service (Core DNS)

Ingress Controller



Ingress Controller

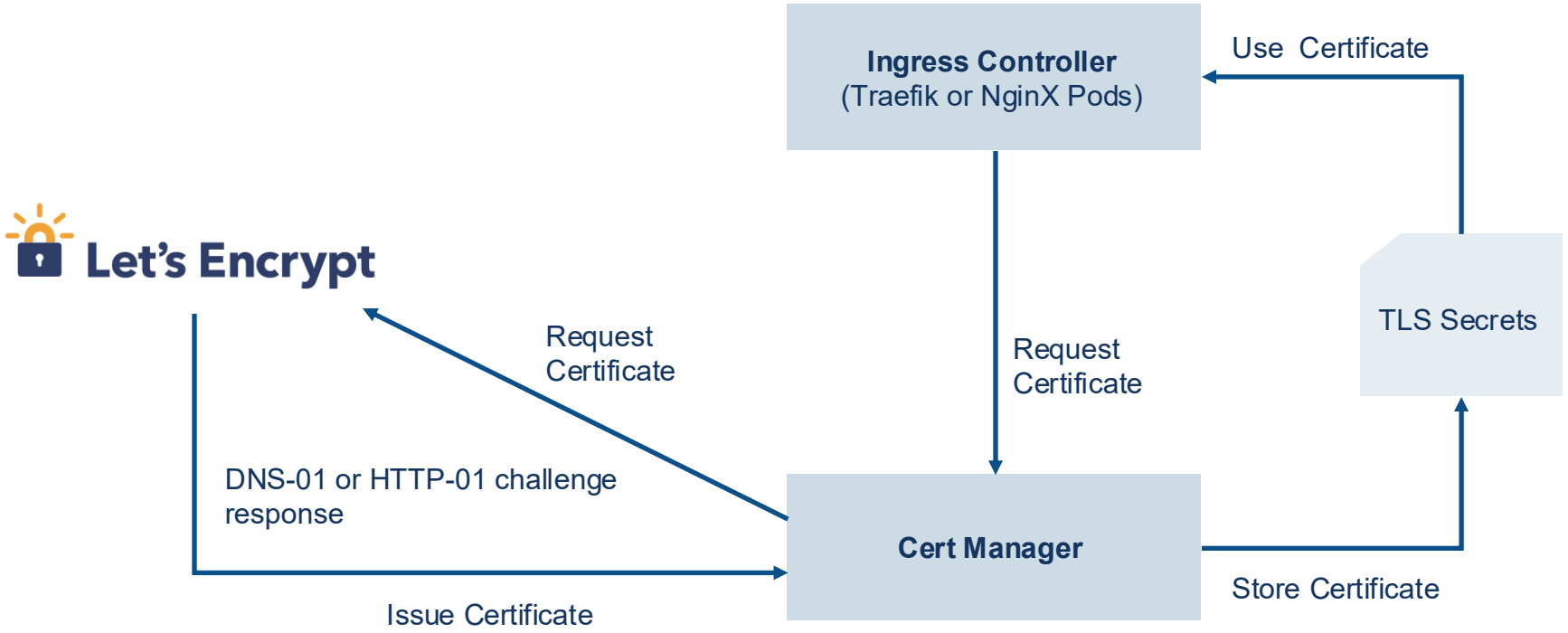


Cert manager

Kubernetes controller that automates certificate management

ACME (Automated Certificate Management Environment)

- Used by CA, e.g. Let's Encrypt, ZeroSSL




```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: wordpress-ingress
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - wordpress.example.com
    secretName: wordpress-tls
  rules:
  - host: wordpress.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: wordpress-service
            port:
              number: 80
```

How a certificate is issued *(using HTTP-01)*

1. *cert-manager* generates a new RSA or ECDSA private key and contacts Let's Encrypt ACME server
2. *Let's Encrypt:* Here is token, please prove that you own the domain by serving the token at http://hello.example.com/.well-known/acme-challenge
3. *cert-manager* spins up a small temporary solver Ingress and Pod that respond at that path
4. *Let's Encrypt* fetches: http://hello.example.com/.well-known/acme-challenge and sees the expected token value
5. *cert-manage*r creates a Certificate Signing Request (CSR) to the ACME server

   Public key (from the generated key pair)
   Signs the CSR with the private key
6. *Let's Encrypt* signs the CSR and issues the certificate
7. *cert-manager* creates/updates a Kubernetes TLS Secret (tls.key and tls.crt)

TLS Handshake

1. User types https://hello.example.com

   Browser initiates TLS handshake with Ingress controller

2. Ingress-controller sends certificate (obtained from Let's Encrypt) to client

   The certificate contains:
   - Domain: wordpress.example.com
   - Public key:
   - Issuer: Let's Encrypt (a trusted CA)
   - Validity period: Usually 90 days
   - Digital signature: Let's Encrypt's signature proving authenticity

3. Browser validates the certificate

   Is it issued by Let's Encrypt? (Trusted CA in browser)
   Does the domain match wordpress.example.com?
   Is it still valid? (Not expired)
   Is the signature valid?

4. Key exchange happens

   Browser generates a random session key (symmetric encryption key, e.g. AES-256)
   Encrypts it with the public key from the certificate
   Sends it to your server
   Your server decrypts it with its private key (stored securely on your server)

Let’s build a Microservice

deployment.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello-world
  labels:
    app: hello-world
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hello-world
  template:
    metadata:
      labels:
        app: hello-world
    spec:
      containers:
      - name: web
        image: johan/helloworld-test
        ports:
        - containerPort: 8080
```

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: hello-world
  annotations:
    cert-manager.io/cluster-issuer: letsencrypt-prod
spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - hello.ltu-m7011e-johan.se
    secretName: hello-world-tls
  rules:
  - host: hello.ltu-m7011e-johan.se
    http:
      paths:
      - path: /
        pathType: Prefix
        backend:
          service:
            name: hello-world-service
            port:
              number: 80
```

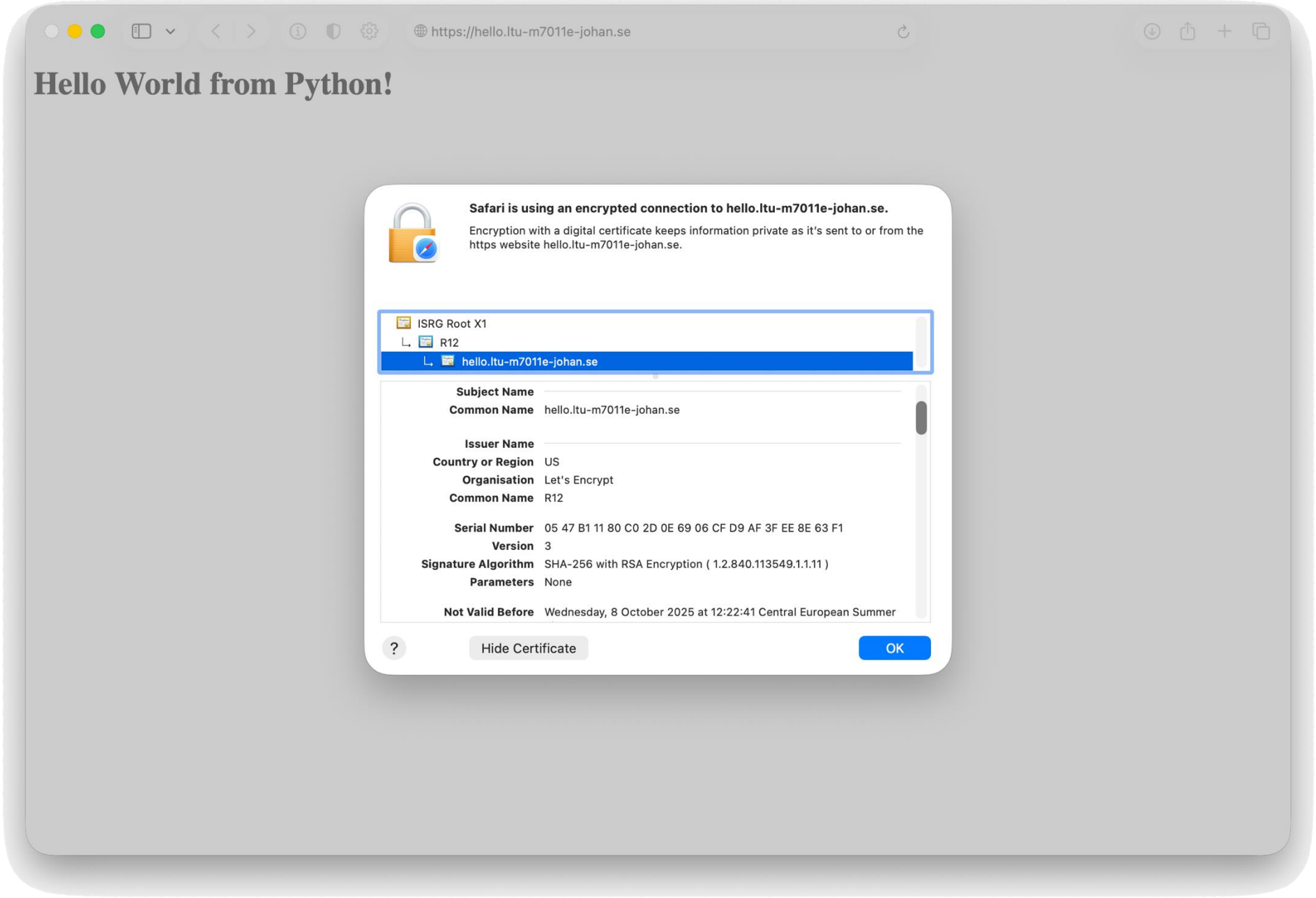
service.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: hello-world-service
spec:
  selector:
    app: hello-world
  ports:
  - protocol: TCP
    port: 80
    targetPort: 8080
```

$ kubectl apply -f deployment.yaml --namespace hello
deployment.apps/hello-world created

$ kubectl apply -f service.yaml --namespace hello
service/hello-world-service created

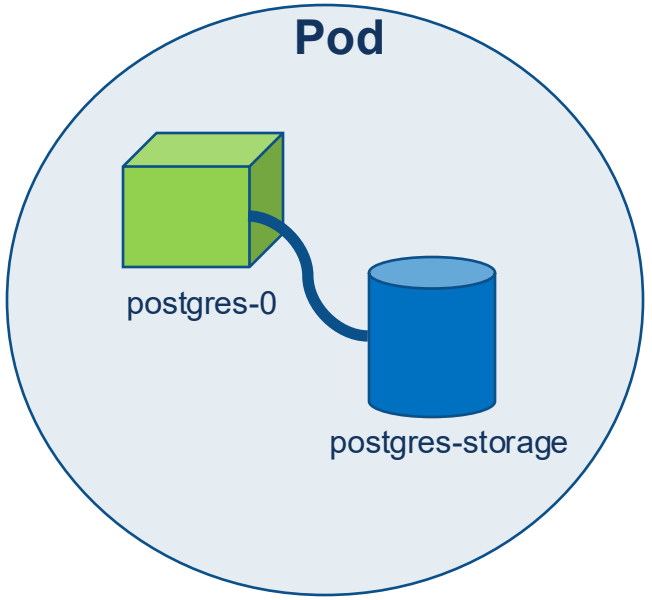
$ kubectl apply -f ingress.yaml --namespace hello
ingress.networking.k8s.io/hello-world created


https://hello.ltu-m7011e-johan.se
Hello World from Python!
Safari is using an encrypted connection to hello.ltu-m7011e-johan.se.
Encryption with a digital certificate keeps information private as it's sent to or from the https website hello.ltu-m7011e-johan.se.
ISRG Root X1
R12
hello.ltu-m7011e-johan.se
Subject Name
Common Name hello.ltu-m7011e-johan.se
Issuer Name
Country or Region US
Organisation Let's Encrypt
Common Name R12
Serial Number 05 47 B1 11 80 C0 2D 0E 69 06 CF D9 AF 3F EE 8E 63 F1
Version 3
Signature Algorithm SHA-256 with RSA Encryption (1.2.840.113549.1.1.11)
Parameters None
Not Valid Before Wednesday, 8 October 2025 at 12:22:41 Central European Summer
?
Hide Certificate
OK

Volumes

Deployments are stateless applications where all pods are identical and interchangeable, while a **StatefulSet** manages stateful applications that need stable pod identities, persistent storage, and ordered startup or shutdown for systems like databases or message queues

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
  namespace: default
spec:
  serviceName: postgres
  replicas: 1
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
      - name: postgres
        image: postgres:15
        ports:
        - containerPort: 5432
          name: postgres
        env:
        - name: POSTGRES_PASSWORD
          value: mysecretpassword
        - name: POSTGRES_DB
          value: mydb
        volumeMounts:
        - name: postgres-storage
          mountPath: /var/lib/postgresql/data
volumeClaimTemplates:
  - metadata:
      name: postgres-storage
    spec:
      accessModes: ["ReadWriteOnce"]
      storageClassName: local-path
      resources:
        requests:
          storage: 1Gi
```



PersistentVolume (PV)
This is the actual piece of storage, like a disk or directory.

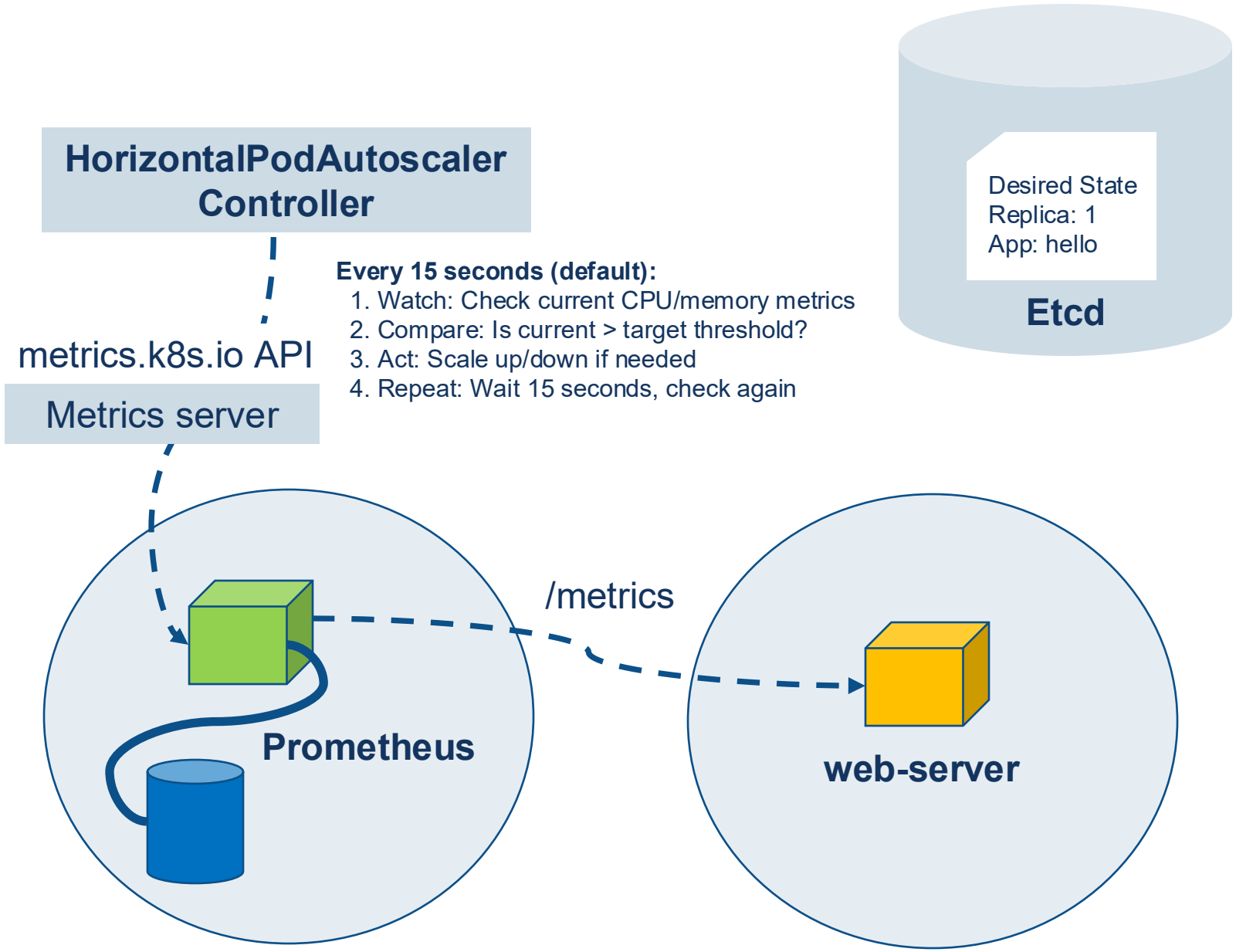
- a local path on a node (local-path),
- a cloud disk (EBS, GCE Persistent Disk),
- or a network storage backend (NFS, Ceph, etc.)

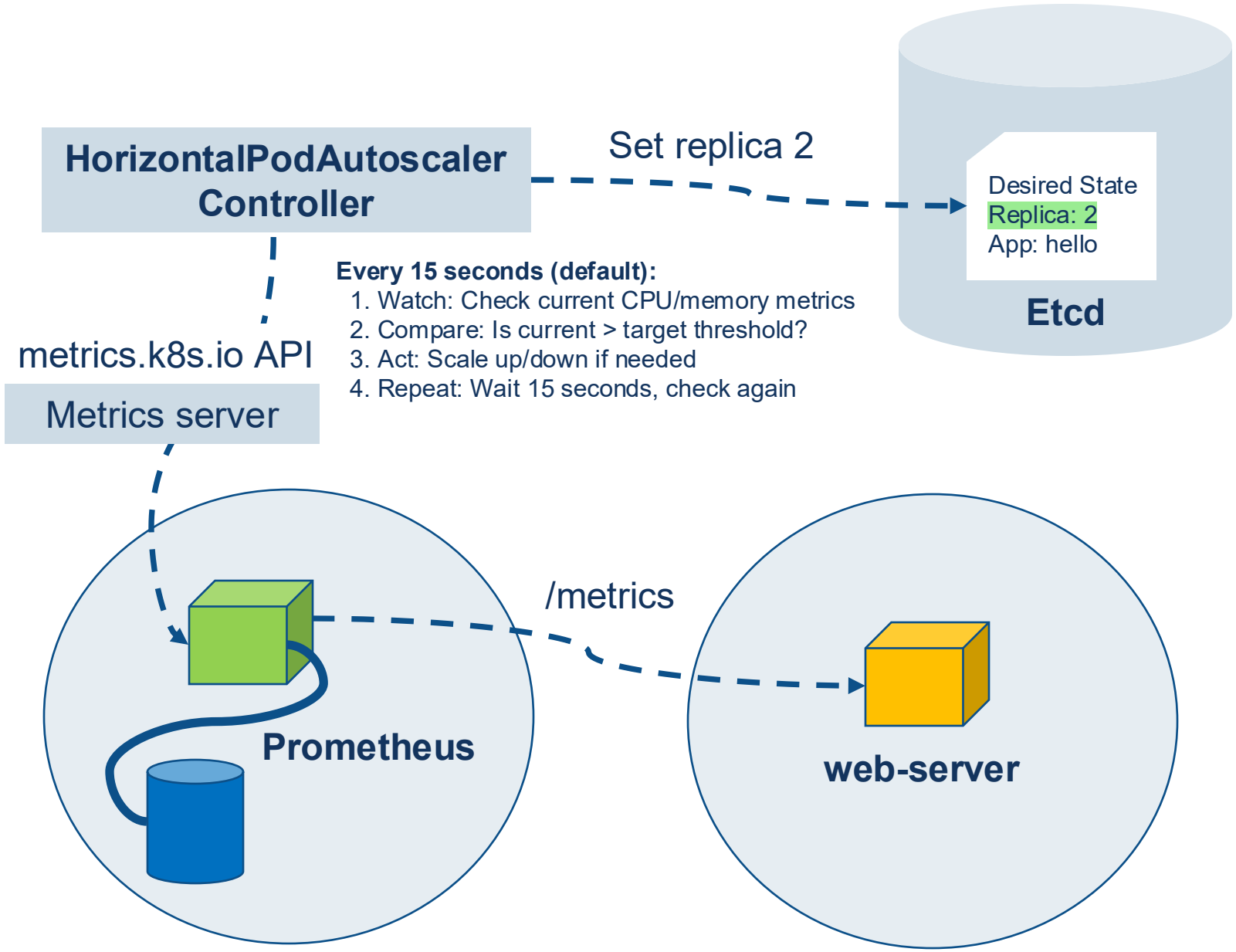
PersistentVolumeClaim (PVC)

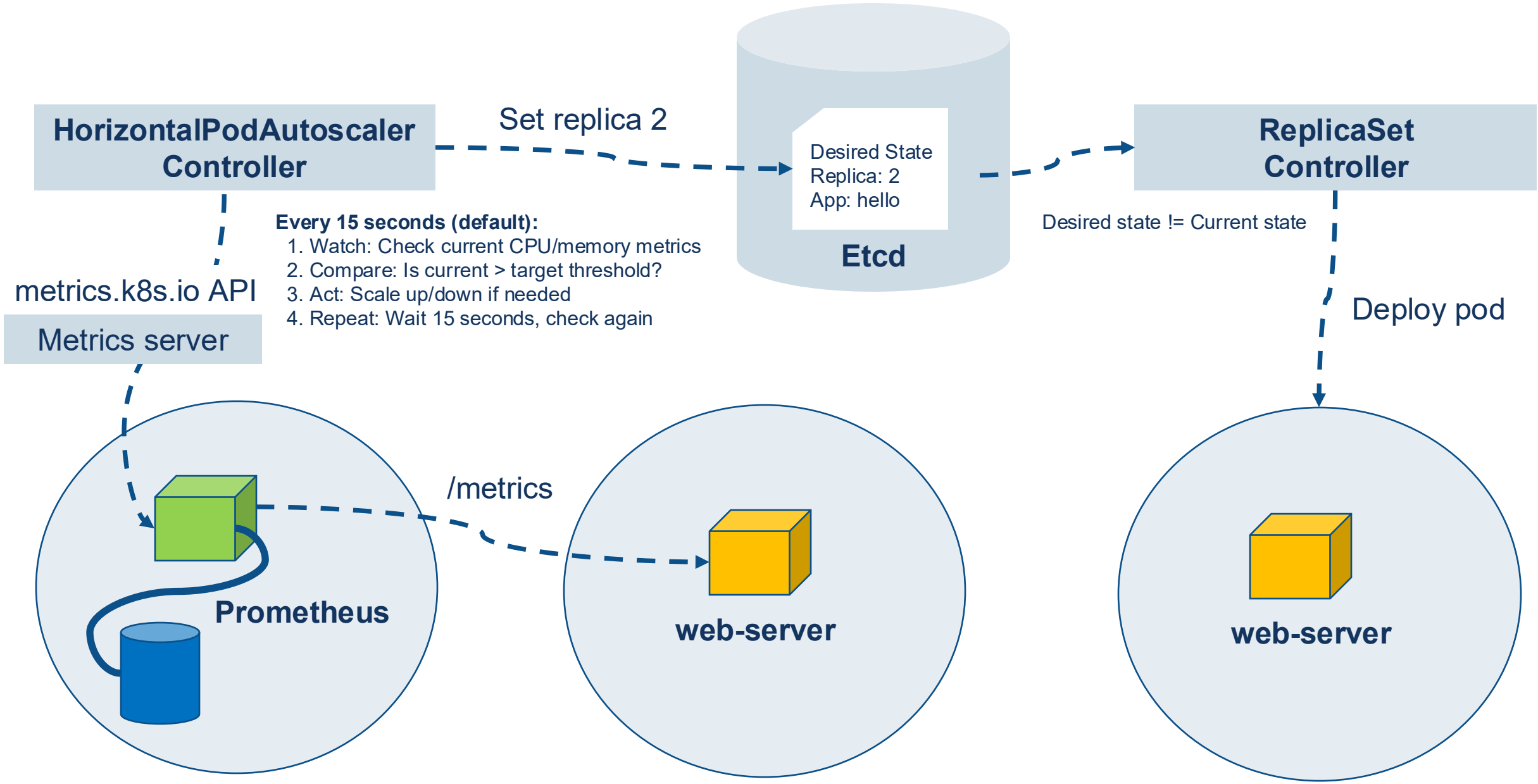
- A PVC is how a pod asks for storage.
- It's a claim, "I need 1Gi of ReadWriteOnce storage from the local-path class.

volumeMount
Inside your Pod spec, the container defines where that volume should appear in its filesystem

volumeClaimTemplates is a template for creating PVCs automatically when the StatefulSet controller spins up pods **volumeClaimTemplates** are processed by the **StatefulSet** controller, not by the pod itself


HorizontalPodAutoscaler Controller
Every 15 seconds (default):
1. Watch: Check current CPU/memory metrics
2. Compare: Is current > target threshold?
3. Act: Scale up/down if needed
4. Repeat: Wait 15 seconds, check again
Desired State
Replica: 1
App: hello
Etcd
metrics.k8s.io API
Metrics server
/metrics
Prometheus
web-server


HorizontalPodAutoscaler Controller
Set replica 2
Desired State
Replica: 2
App: hello
Etcd
Every 15 seconds (default):
1. Watch: Check current CPU/memory metrics
2. Compare: Is current > target threshold?
3. Act: Scale up/down if needed
4. Repeat: Wait 15 seconds, check again
metrics.k8s.io API
Metrics server
/metrics
Prometheus
web-server


HorizontalPodAutoscaler Controller
Set replica 2
Desired State
Replica: 2
App: hello
Etcd
ReplicaSet Controller
Every 15 seconds (default):
1. Watch: Check current CPU/memory metrics
2. Compare: Is current > target threshold?
3. Act: Scale up/down if needed
4. Repeat: Wait 15 seconds, check again
Desired state != Current state
metrics.k8s.io API
Metrics server
Deploy pod
/metrics
Prometheus
web-server
web-server

Autoscaling

autoscaler.yaml

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: hello-world-hpa
  namespace: hello
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: hello-world
  minReplicas: 1
  maxReplicas: 10
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 40
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0    # Scale up
      policies:
      - type: Percent
        value: 100                     # Double pods if needed
        periodSeconds: 1               # Check every 1 second
      selectPolicy: Max
    scaleDown:
      stabilizationWindowSeconds: 0    # Wait only 15 seconds (was 60)
      policies:
      - type: Pods
        value: 5                       # Remove 5 pods at a time
        periodSeconds: 1               # Every  second
      selectPolicy: Max                # Scale down
```

app.py

```python
from flask import Flask
import hashlib
import time
import logging
import sys

app = Flask(__name__)
request_count = 0

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(message)s',
    stream=sys.stdout
)
logger = logging.getLogger(__name__)


def cpu_intensive_work():
    """Burn CPU to trigger autoscaling"""
    result = ""
    for i in range(500000):
        result = hashlib.sha256(f"dummy-work-{i}".encode()).hexdigest()
    return result

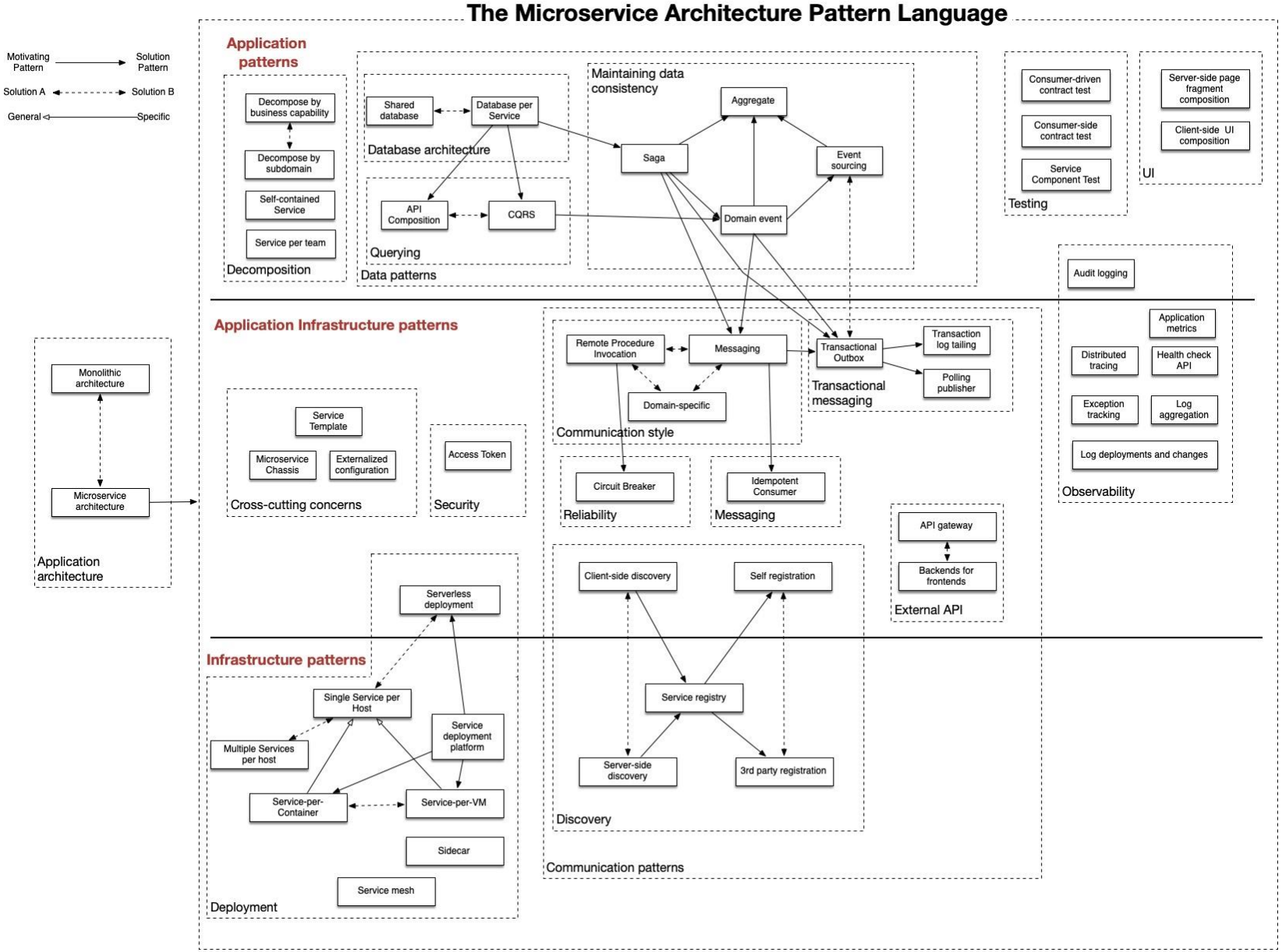
@app.route('/')
def hello():
    global request_count
    request_count += 1

    # Measure CPU work time
    start_time = time.time()
    hash_result = cpu_intensive_work()
    cpu_time = time.time() - start_time

    logger.info(f"REQUEST #{request_count} - CPU time: {cpu_time:.3f}s - Hash: {hash_result[:8]}")

    return '''
    <p>Request count: {}</p>
    <p>CPU work time: {:.3f} seconds</p>
    <p>Hash computed: {}...</p>
    '''.format(request_count, cpu_time, hash_result[:16])

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=8080)
```



Copyright © 2024. Chris Richardson Consulting, Inc. All rights reserved.

Learn-Build-Assess Microservices http://adopt.microservices.io

https://microservices.io/patterns/index.html

Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication, providing traffic control, security (mTLS), and observability through sidecar proxies and a centralised control plane, without changing application code

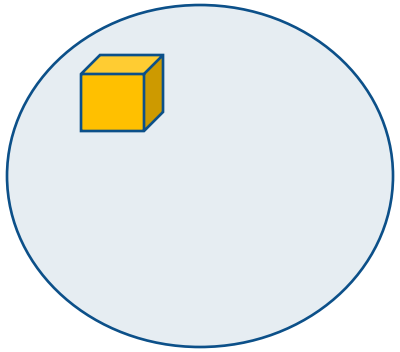
A service mesh moves several concerns out of application code and into the infrastructure, *solving the complexity of modern distributed systems without touching your application code*

Use cases

- **Zero-trust networking:** automatically encrypt and authenticate service-to-service traffic with mTLS
- **Traffic shaping:** rate control, route a small percentage of traffic to a new version for canary or blue/green deployments
- **Resilience:** implement retries, circuit breaking, and failover without code changes
- **Observability:** collect metrics, logs, and distributed traces for every request path
- **Policy enforcement:** define and enforce “who can talk to whom” at the network layer

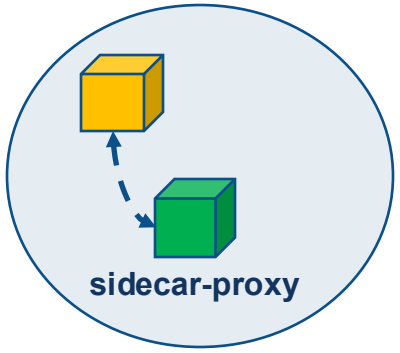
Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.



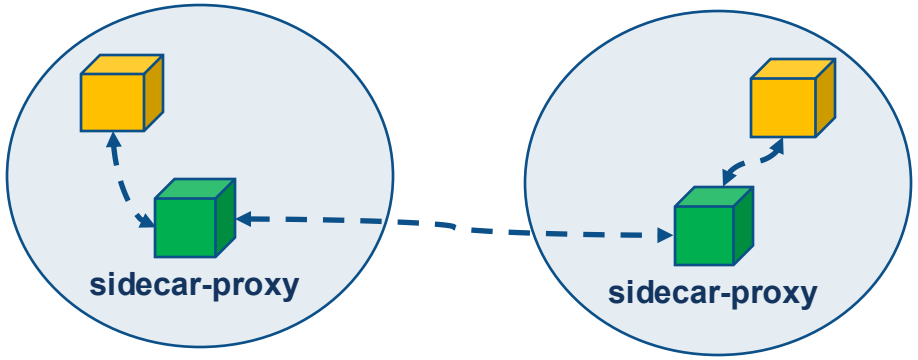
Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.



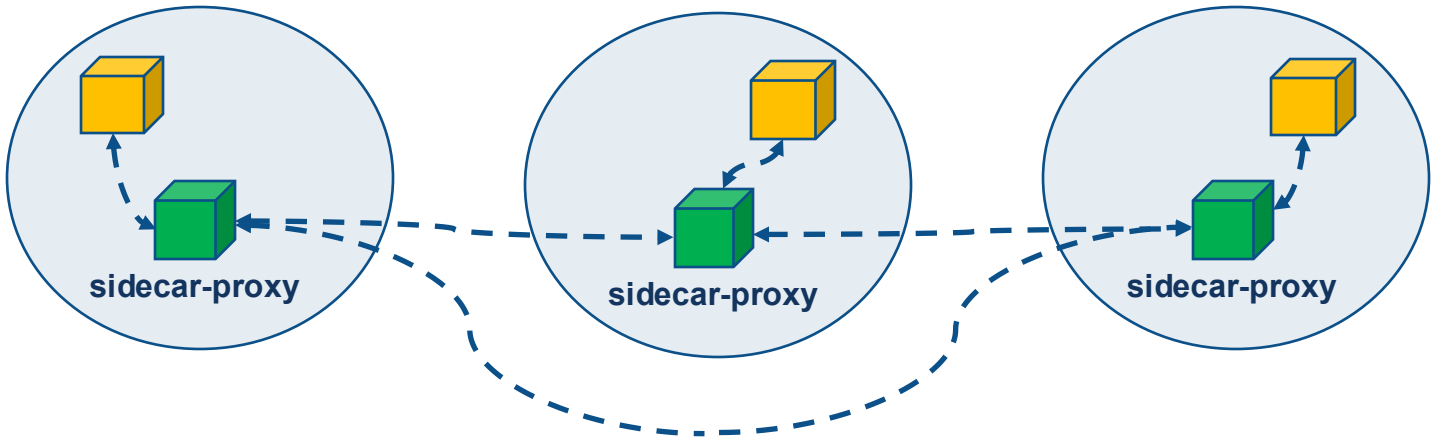
Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.



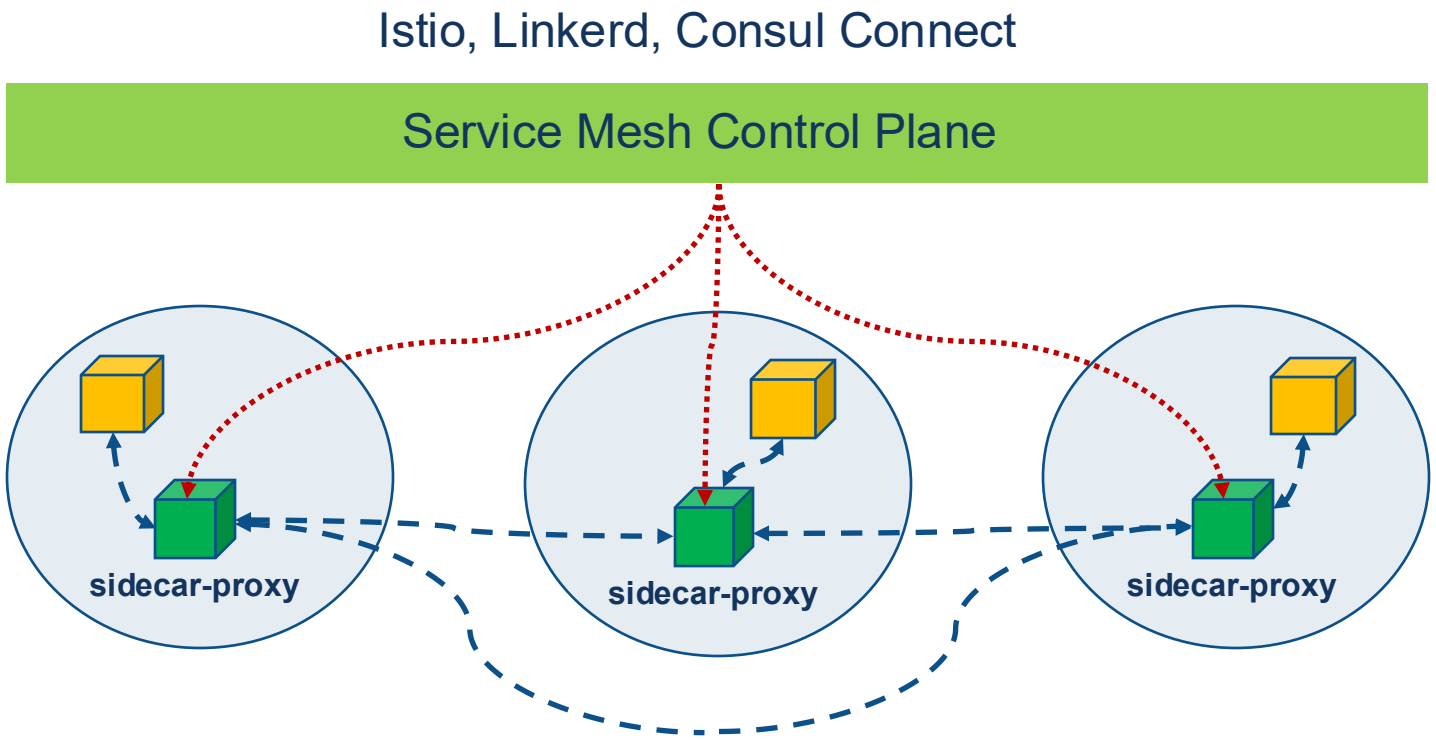
Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.



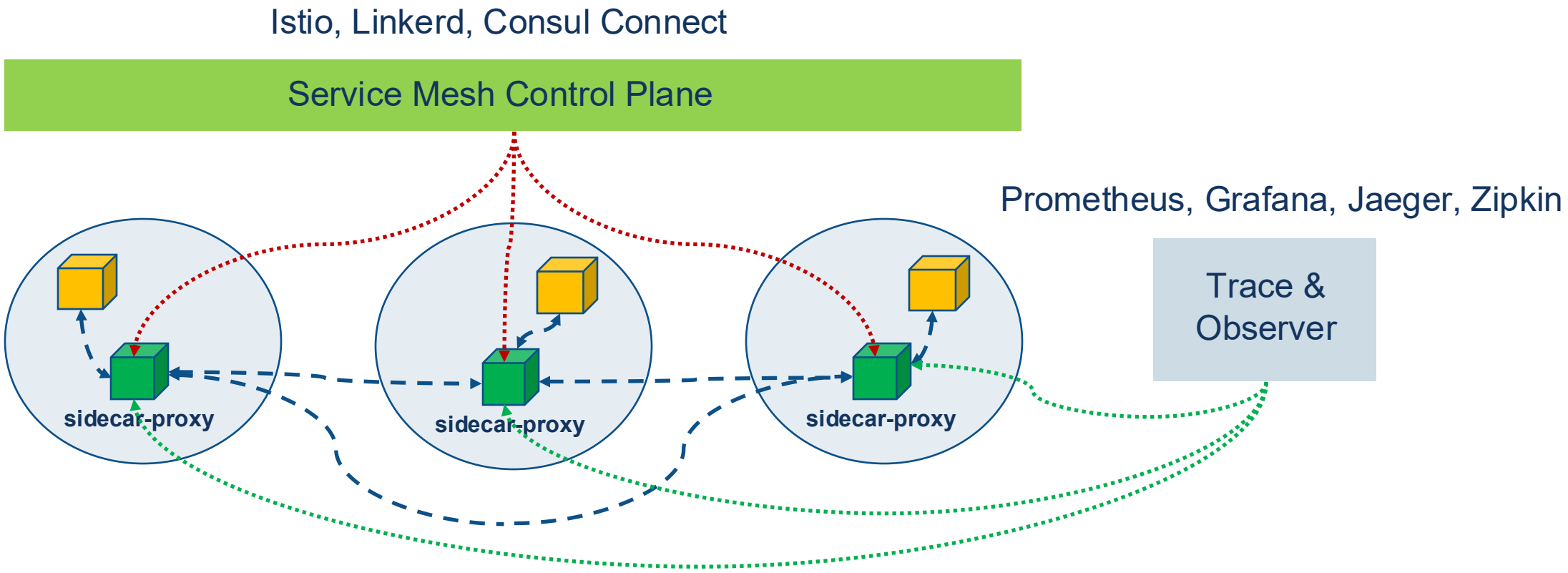
Service Meshes

Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.



Service Meshes

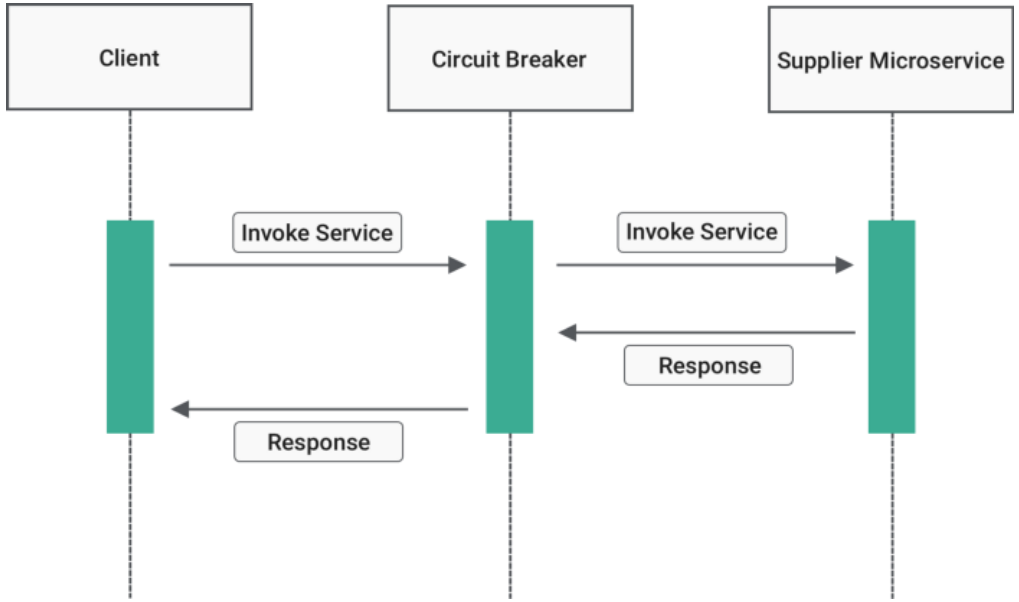
Service Mesh is an infrastructure layer that manages service-to-service communication — providing traffic control, security (mTLS), and observability through sidecar proxies and a centralized control plane, without changing application code.

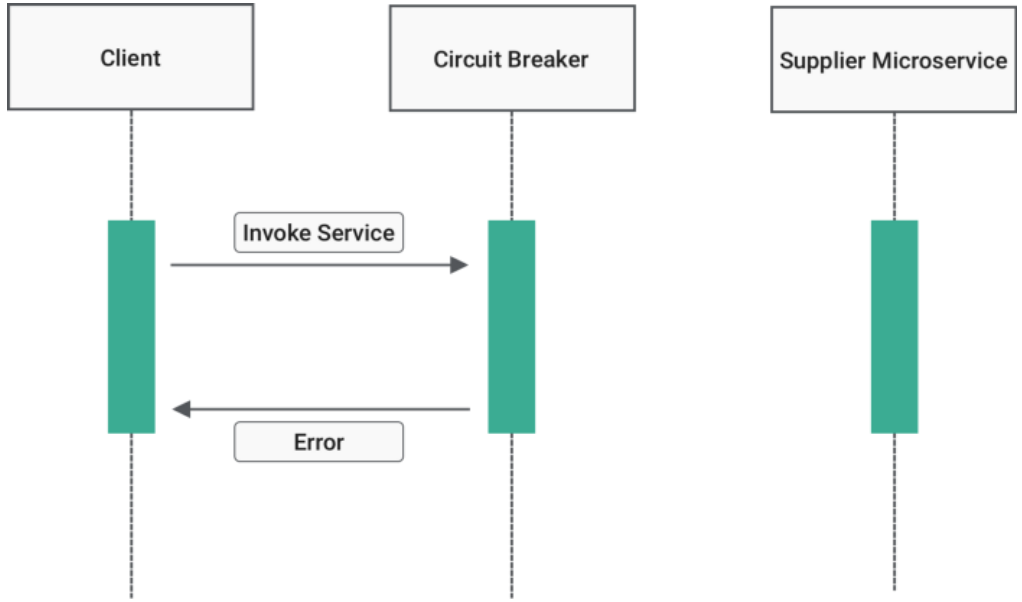


Circuit breakers

In a microservices deployment, there might be hundreds of services calling the same backend
These services could implement their own failure detection differently, with different thresholds, timeouts, retry logic, etc.

A circuit breaker **fails fast** and it returns an immediate error (“Service temporarily unavailable”) instead of waiting for network timeouts, **thus avoiding cascading failures** across services





By using a shared circuit breaker (e.g. service mesh sidecar proxy), you get consistent, declarative behaviour

Operators

An Operator is a custom controller that knows how to manage a specific kind of application or resource, e.g., databases, message queues, or storage systems

- Extend the Kubernetes API to understand new resource types
- CRD: teaches Kubernetes a new type (EdgeDevice).
- Custom Resource: defines the desired device state.
- Controller: watches those CRs and acts to match reality

edgedevice.yaml

```yaml
apiVersion: iot.example.com/v1
kind: EdgeDevice
metadata:
  name: camera-01
spec:
  firmwareVersion: "3.2"
  config:
    resolution: "1080p"
    sampleRate: "5s"
```

edgedevice_crd.yaml

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: edgedevices.iot.example.com
spec:
  group: iot.example.com
  scope: Namespaced
  names:
    kind: EdgeDevice
    plural: edgedevices
  versions:
  - name: v1
    served: true
    storage: true
    schema:
      openAPIV3Schema:
        type: object
        properties:
          spec:
            type: object
            properties:
              firmwareVersion:
                type: string
              config:
                type: object
                properties:
                  resolution:
                    type: string
                  sampleRate:
                    type: string
```

Operators

An Operator is a custom controller that knows how to manage a specific kind of application or resource, e.g., databases, message queues, or storage systems

- Extend the Kubernetes API to understand new resource types.

operator.yaml

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: edge-device-operator
spec:
  replicas: 2
  template:
    spec:
      containers:
      - name: operator
        image: myrepo/edge-device-operator:latest
```

edgedevice_operator.go

```go
mgr, err := ctrl.NewManager(ctrl.GetConfigOrDie(), ctrl.Options{
    LeaderElection:   true,
    LeaderElectionID: "edge-device-operator-leader",
})

…

// Pseudo-Operator: keeps IoT device config in sync
for {
    desired := getDesiredConfig("camera-01")   // from K8s spec
    actual  := getDeviceConfig("camera-01")    // from device API

    if desired != actual {
        fmt.Println("Updating device to match desired config...")
        pushConfigToDevice("camera-01", desired)
        updateStatus("camera-01", desired)     // patch status in K8s
    }

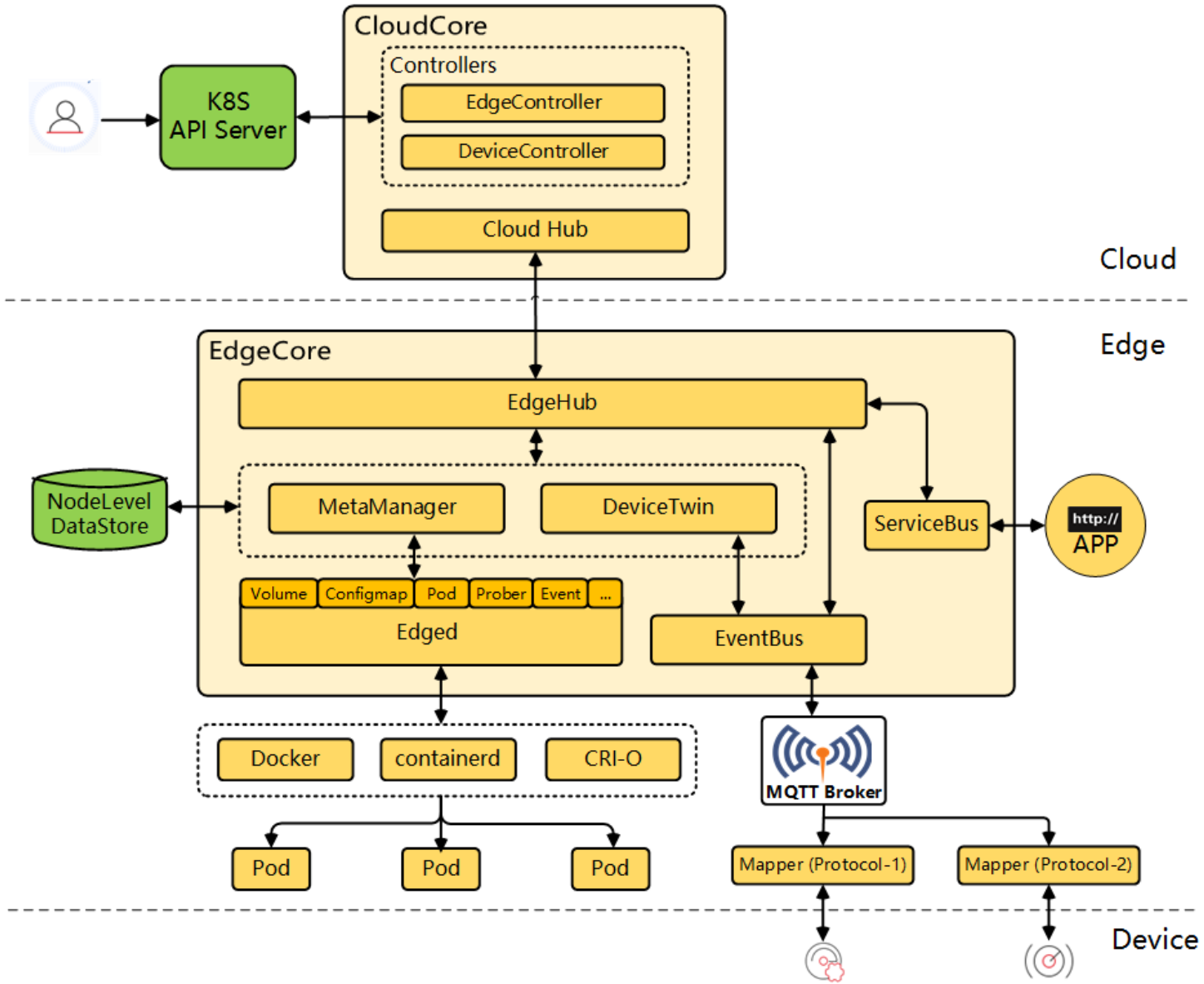
    time.Sleep(10 * time.Second)
}
```

EdgeKube

KubeEdge extends Kubernetes from the cloud to the edge

Main idea: Still use Kubernetes control plane (API server, etcd, controllers), but introduce edge nodes that run workloads and manage IoT or edge devices even when disconnected

- CloudHub watches for Pod and Device changes in etcd and communicates with edges
- EdgeHub keeps a connection (MQTT/WebSocket) to CloudHub
- DeviceTwin maintains a digital twin of each physical IoT device (its state, config, metadata)



Separation of Concerns
The Hidden Architecture of Modern Systems

Microservices split big systems into small, independent components, ***but bring operational complexity in deployment, networking, and security***

Kubernetes manages infrastructure concerns: scheduling, scaling, recovery, and configuration so developers don't manage servers, ***only desired state***.

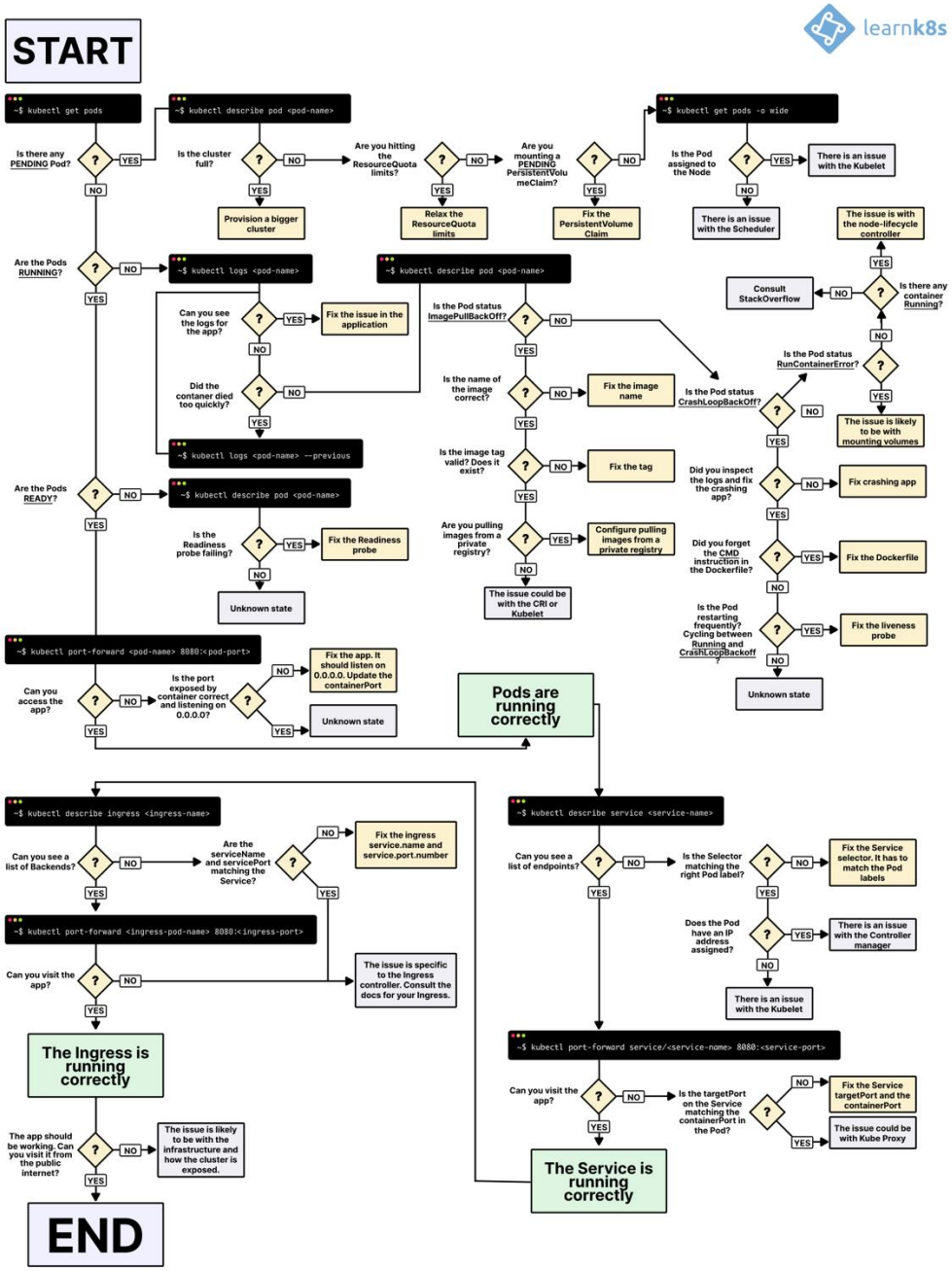
Service Meshes manage communication concerns: traffic routing, retries, observability, and encryption so developers ***don't need to manage networking, only business logic***

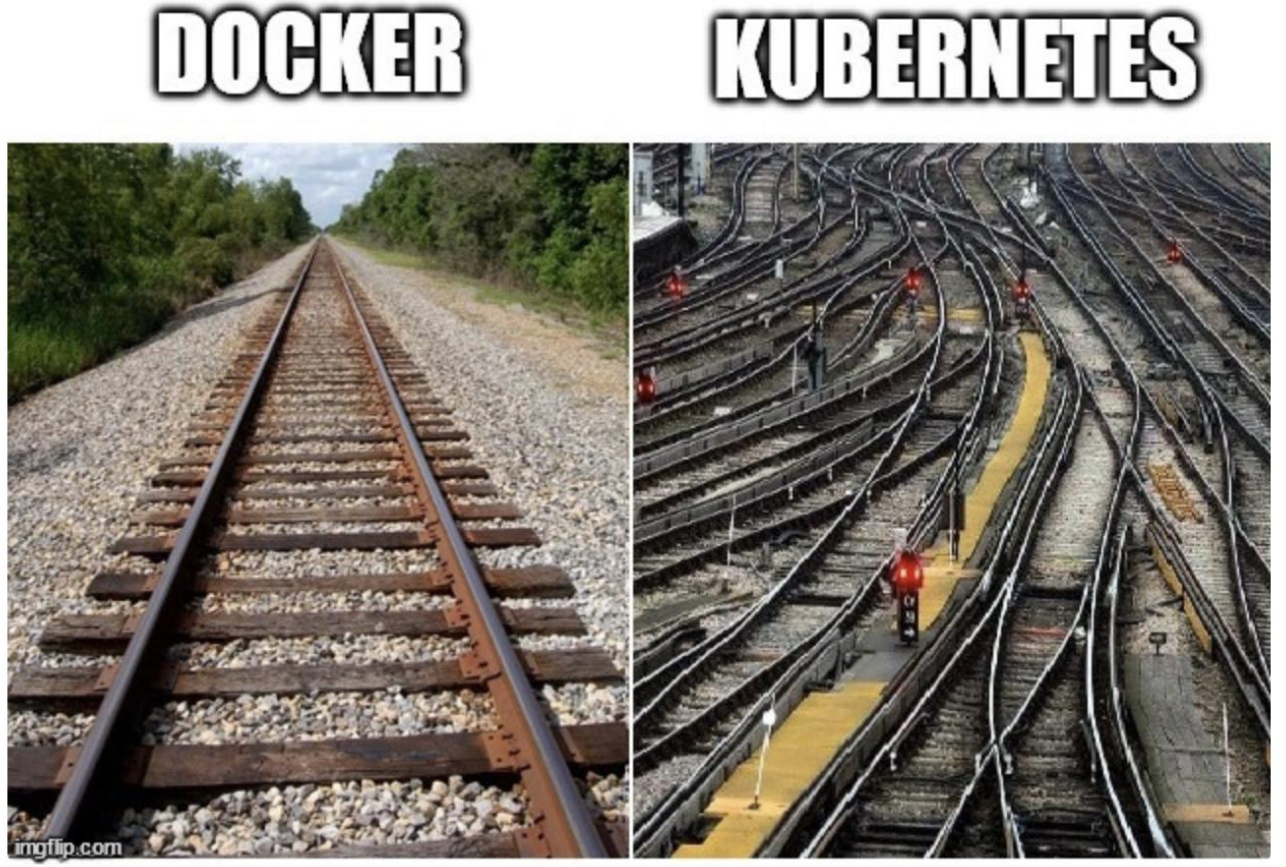
Together they enforce separation of concerns:

- Developers write code and define intent
- The platform automates resilience, security, and operations

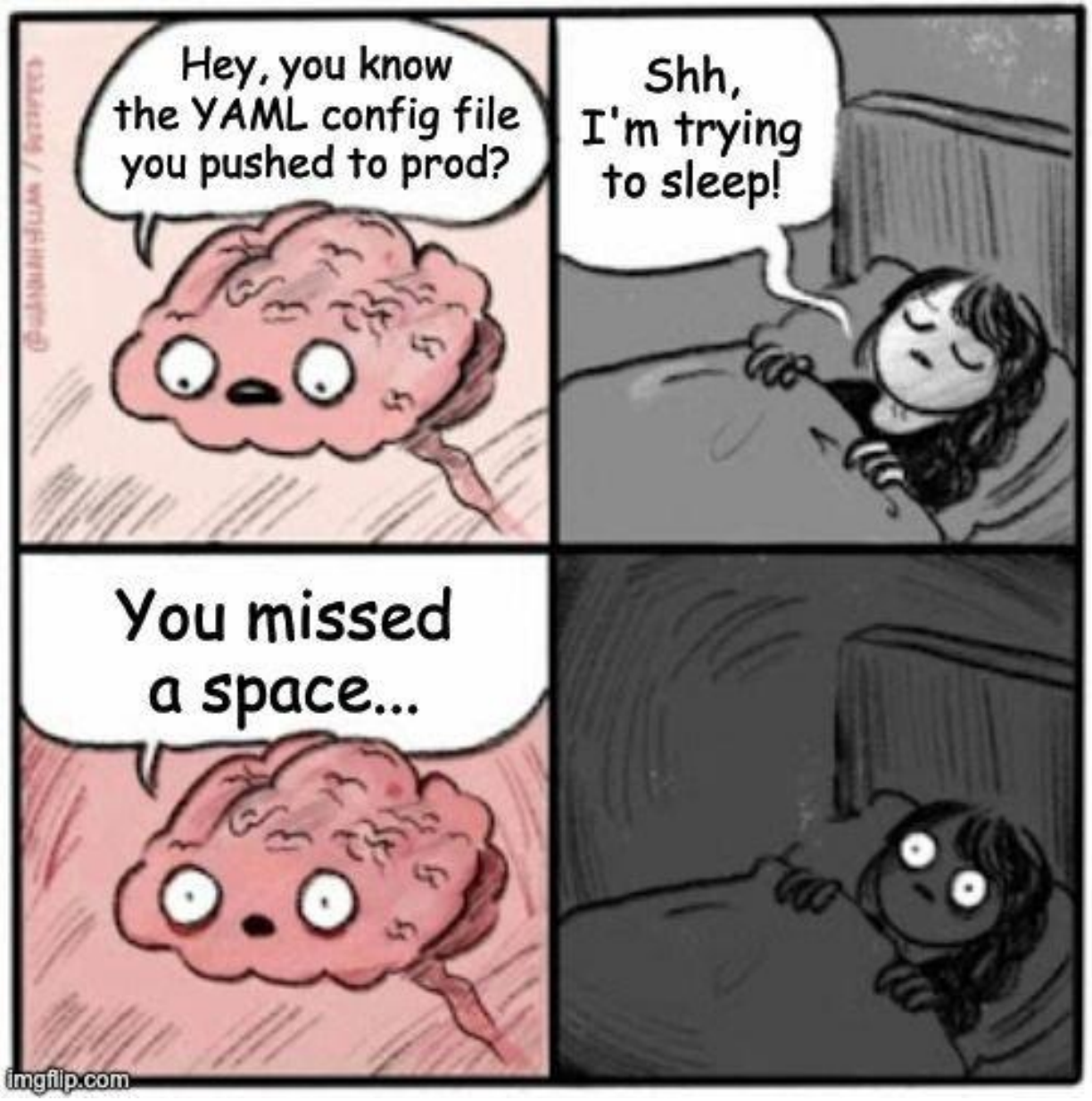
All distributed systems have trade-offs!
(the main theme of the course)

But, what is the trade-off of Kubernetes?





https://dev.to/tythos/accessible-kubernetes-with-terraform-and-digitalocean-12o



https://community.cisco.com/t5/the-break-room/meme-monday-yaml-spacing/td-p/4909223


RE.VA
MONRC.VIA
msc
RESCUE

Challenges with Kubernetes

Complexity Explosion

- **Deep learning curve**: Steep climb from Docker to K8s mastery
- **YAML hell**: Managing hundreds of configuration files
- **Too many choices**: Overwhelming ecosystem (100+ storage drivers, 50+ networking plugins)
- **Abstraction overload**: Pods, Services, Ingress, ConfigMaps, Secrets, etc.
- **Debugging difficulty**: Distributed systems are inherently hard to troubleshoot

Operational Challenges

- **Upgrade nightmares**: K8s cluster upgrades can break everything
- **Multi-cluster management**: Managing multiple K8s clusters is painful
- **Monitoring complexity**: Need specialized tools (Prometheus, Grafana, Jaeger)

Challenges with Kubernetes

Security Concerns

- **Shared kernel risks**: Container escape vulnerabilities affect all containers
- **Image vulnerabilities**: Pulling unverified images from Docker Hub, compromised base images affect many
- **Complex RBAC**: Kubernetes permissions are hard to get right
- **Network policies**: Difficult to properly segment container traffic
- **Secrets management**: Storing sensitive data securely is difficult

Development Friction

- **Cognitive overload**: Developers need to understand infrastructure, : Why is this so complicated?
- **Skills gap**: Not enough Kubernetes experts
- **Tool fatigue**: Constant stream of new "cloud-native" tools, CI/CD, service mesh, monitoring - too many tools
- **Configuration drift**: Dev, staging, prod environments diverge
- **Over-engineering**: Using containers where simple solutions works

Serverless/Function-as-a-Service Era (2014 - Present)

Serverless means developers write functions/applications without thinking about servers, containers, or infrastructure at all, ***the cloud provider handles everything***

Changes

- **No infrastructure management**: Zero servers, VMs, or containers to manage
- **True pay-per-use**: Pay only for actual execution time (milliseconds)
- **Improved auto-scaling**: 0 to 10,000 instances instantly
- **Event-driven by default**: Triggered by HTTP, queues, streams, schedules
- **Developer focus**: Write business logic only

Serverless/Function-as-a-Service Era (2014 - Present)

New Paradigms

- **Functions as deployment unit**: Deploy individual functions
- **Stateless by design**: Functions must be idempotent
- **Event-driven architecture**: Everything is an event
- **Managed services composition**: Glue together cloud services
- **GitOps for functions**: Push code, platform does the rest

Serverless/Function-as-a-Service Era (2014 - Present)

Commerical

- AWS Lambda
- Azure Functions
- Google Cloud Functions
- AWS Fargate
- Google Cloud Run
- Azure Container Instances

Open Source

- OpenFaaS
- Apache OpenWhisk
- Faasd
- Knative

Amazon Lambda

faas.py

```
def lambda_handler(event, context):
    return {"message": "Hello from a container!"}
```

```
$ aws ecr create-repository --repository-name hello-lambda
$ aws ecr get-login-password | docker login --username AWS --password-stdin <account-id>.dkr.ecr.<region>.amazonaws.com
$ docker tag hello-lambda:latest <account-id>.dkr.ecr.<region>.amazonaws.com/hello-lambda:latest
$ docker push <account-id>.dkr.ecr.<region>.amazonaws.com/hello-lambda:latest

$ aws lambda create-function \
  --function-name hello-lambda-container \
  --package-type Image \
  --code ImageUri=<account-id>.dkr.ecr.<region>.amazonaws.com/hello-lambda:latest \
  --role arn:aws:iam::<account-id>:role/<LambdaExecutionRole>

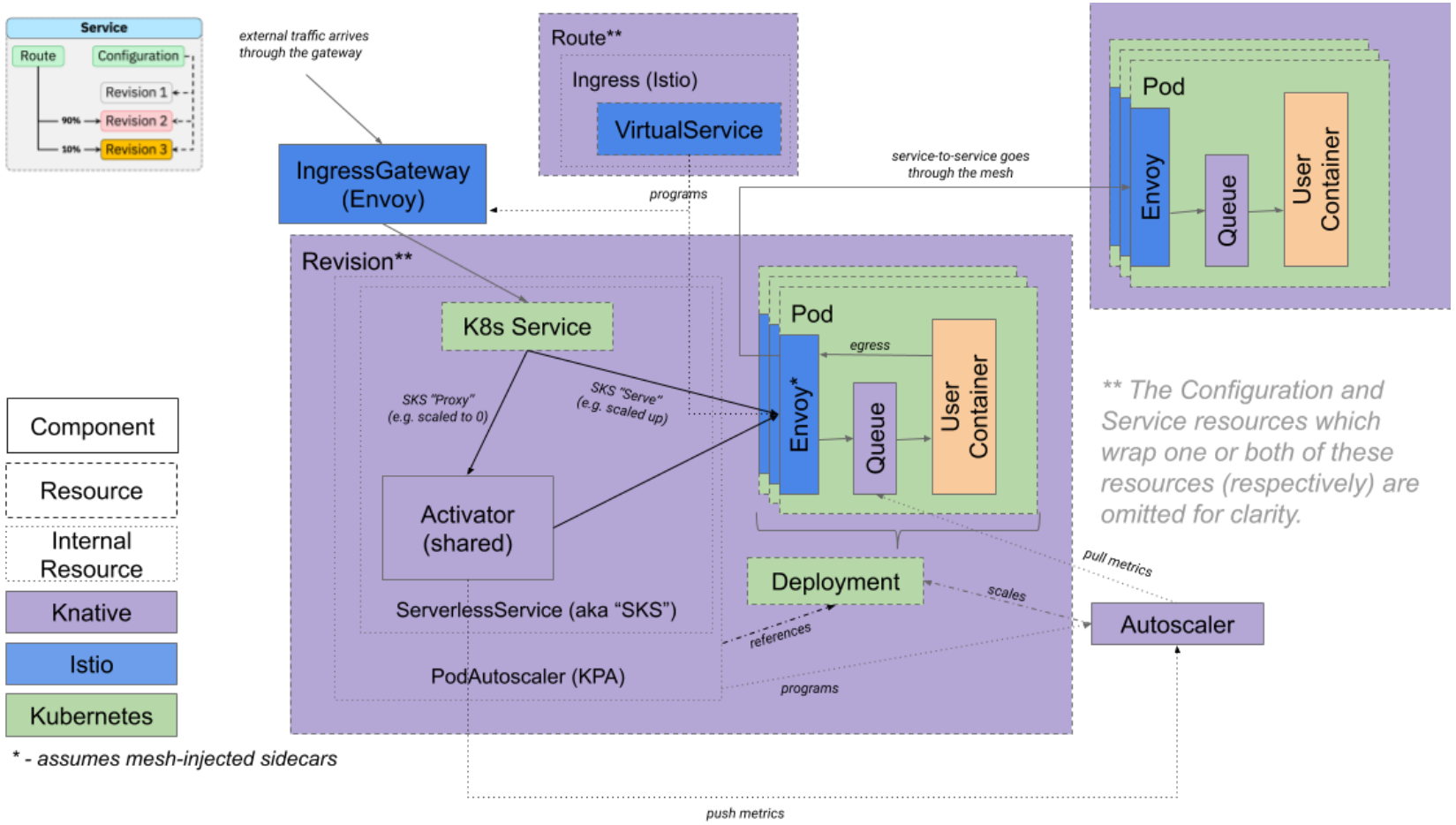
$ aws lambda invoke \
  --function-name hello-lambda-container \
  --payload '{"name":"Alice"}' \
  output.json
cat output.json
```

Knative (Serverless on Kubernetes)

Knative is an open-source Kubernetes-based serverless platform

Run ***function-as-a-service*** on Kubernetes with automatic scaling without manually managing Deployments, Services, or networking

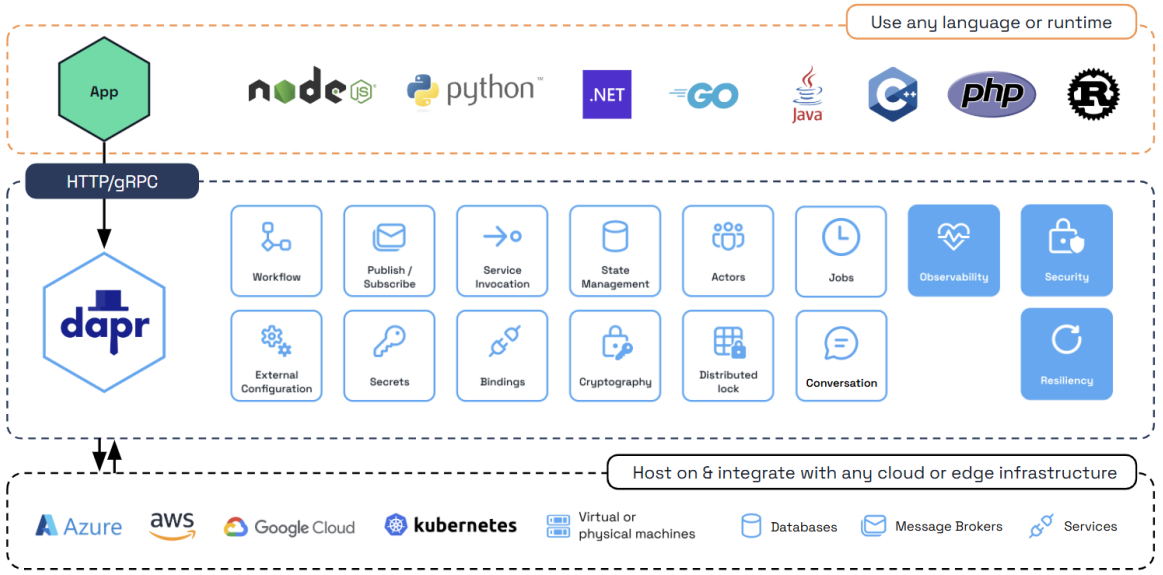
https://knative.dev/docs



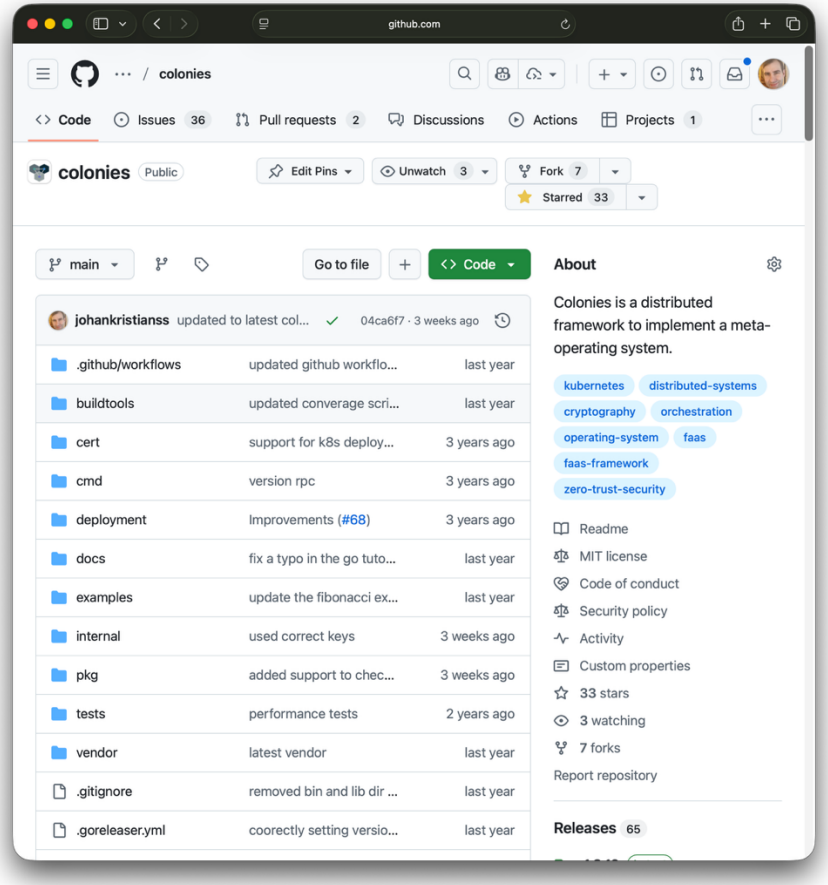
Dapr – Distributed Application Runtime

"With the current wave of cloud adoption, web, database architectures are trending more toward microservice application architectures, which are inherently distributed. You shouldn't have to become a distributed systems expert just to create microservices application"

- Works on any Kubernetes, VM, or edge device
- Removes repetitive infrastructure code from every service
- Complements frameworks like Knative, KubeEdge, or even Service Meshes



ColonyOS – Meta-Operating Systems
Edge-Cloud Computing Continuums



https://github.com/colonyos/colonies



https://colonyos.io

Evolutionary Trends

From Monoliths to Computing Continuums

- Each paradigm shift abstracts more infrastructure complexity and pushes automation deeper into the platform
- We are moving from manual integration to self-managing systems

What Drives This Evolution?

- Abstraction & Automation – Reduce operational friction; platforms manage what humans used to
- Developer Focus – Shift from managing servers to writing business logic
- Distribution & Connectivity – Everything talks to everything, everywhere
- Portability & Consistency – Build once, run anywhere, now including cloud, edge, and device
- Intelligence at the Edge – Compute is moving closer to data; autonomy replaces central control


LULEÅ
UNIVERSITY
OF TECHNOLOGY